Universidade de Brasilia

Brasília Military Middle School 2017

Arthur Botelho, Henrique Quirino, Luiz Eduardo Reis

2026-08-27

Contest (1)

template.cpp 28 lines

```
#include <bits/stdc++.h>
using namespace std;
// Ours
#define endl '\n'
#define debug(x...) cout<<"["#x"]: ",[](auto...$){((cout<<$<<";
    "),...);}(x),cout<<endl
// KACTL
typedef long long ll;
typedef long double ld;
#define sz(x) (int)x.size()
#define all(x) x.begin(), x.end()
#define vi vector<int>
#define pii pair<int, int>
#define rep(i, a, b) for(int i = (a); i < (b); i++)
#define pb push_back
#define eb emplace_back

constexpr int oo = (((unsigned int)-1)>>2);

void solve(){
  int n; cin >> n;
}

int main() {
  cin.tie(0)->sync_with_stdio(0);
  int t = 1;
  // cin >> t;
  while (t--) solve();
}
```

.bashrc 21 lines

```
teste() {
    for f in "$2"/*; do
        echo "=== $f"
        cat $f
        echo "=== Out"
        $1 < $f
    done
}
# teste ./b bf
stress() {
    for ((i=1; ; i++)) do
        echo "Test $i"
        echo $i | $1 > gen_in
        $2 < gen_in > main_out
        $3 < gen_in > brute_out
        if ! diff -w main_out brute_out > /dev/null; then
            return 1
        fi
    done
}
# stress "python3 gen.py" ./main ./brute
```

.vimrc 5 lines

```
set autoindent smartindent ts=4 sw=4 rnu nu
" Select region and then type :Hash to hash your selection.
" Useful for verifying that there aren't mistypes.
ca Hash w !cpp -dD -P -fpreprocessed \| tr -d '[:space:]' \
 \| md5sum \| cut -c-6
```

hash.sh 3 lines

```
# Hashes a file, ignoring all whitespace and comments. Use for
# verifying that code was correctly typed.
cpp -dD -P -fpreprocessed | tr -d '[:space:]'| md5sum |cut -c-6
```

Makefile 1 lines

```
CXXFLAGS := -std=c++20 -Wall -Wextra -Wshadow -Wconversion -
    fsanitize=address,undefined -g3
```

Theory (2)

2.1 Geometry

2.1.1 Trigonometry

$$(V+W)\tan(v-w)/2 = (V-W)\tan(v+w)/2$$

where V, W are lengths of sides opposite angles v, w.

$$a\cos x + b\sin x = r\cos(x-\phi)$$
$$a\sin x + b\cos x = r\sin(x+\phi)$$

where $r = \sqrt{a^2+b^2}, \phi = \text{atan2}(b, a)$.

2.1.2 Triangles

Length of median (divides triangle into two equal-area triangles): $m_a = \frac{1}{2}\sqrt{2b^2+2c^2-a^2}$

Length of bisector (divides angles in two):

$$s_a = \sqrt{bc\left[1-\left(\frac{a}{b+c}\right)^2\right]}$$

Law of tangents: $\dfrac{a+b}{a-b} = \dfrac{\tan\frac{\alpha+\beta}{2}}{\tan\frac{\alpha-\beta}{2}}$

2.1.3 Quadrilaterals

With side lengths a, b, c, d, diagonals e, f, diagonals angle θ, area A and magic flux $F = b^2+d^2-a^2-c^2$:

$$4A = 2ef\cdot\sin\theta = F\tan\theta = \sqrt{4e^2f^2-F^2}$$

For cyclic quadrilaterals the sum of opposite angles is 180°, $ef = ac+bd$, and $A = \sqrt{(p-a)(p-b)(p-c)(p-d)}$ (p is semiperimeter).

2.2 Combinatorics

2.2.1 Permutations

Cycles

Let $g_S(n)$ be the number of n-permutations whose cycle lengths all belong to the set S. Then

$$\sum_{n=0}^{\infty} g_S(n)\frac{x^n}{n!} = \exp\left(\sum_{n\in S}\frac{x^n}{n}\right)$$

Derangements

Permutations of a set such that none of the elements appear in their original position.

$$D(n) = (n-1)(D(n-1)+D(n-2)) = nD(n-1)+(-1)^n = \left\lfloor\frac{n!}{e}\right\rceil$$

2.2.2 Partitions and subsets

Partition function

Number of ways of writing n as a sum of positive integers, disregarding the order of the summands.

$$p(0)=1,\ p(n) = \sum_{k\in\mathbb{Z}\setminus\{0\}}(-1)^{k+1}p(n-k(3k-1)/2)$$

$$p(n)\sim 0.145/n\cdot\exp(2.56\sqrt{n})$$

n	0	1	2	3	4	5	6	7	8	9	20	50	100
$p(n)$	1	1	2	3	5	7	11	15	22	30	627	~2e5	~2e8

Lucas' Theorem

Let n, m be non-negative integers and p a prime. Write $n = n_kp^k+\ldots+n_1p+n_0$ and $m = m_kp^k+\ldots+m_1p+m_0$. Then $\binom{n}{m} \equiv \prod_{i=0}^{k}\binom{n_i}{m_i} \pmod p$.

2.2.3 General purpose numbers

Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t-1}$ (FFT-able).
$B[0,\ldots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \ldots]$

Sums of powers:

$$\sum_{i=1}^{n} i^m = \frac{1}{m+1}\sum_{k=0}^{m}\binom{m+1}{k}B_k\cdot(n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^{\infty} f(i) = \int_m^{\infty} f(x)dx - \sum_{k=1}^{\infty} \frac{B_k}{k!} f^{(k-1)}(m)$$
$$\approx \int_m^{\infty} f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

Stirling numbers of the first kind

Number of permutations on n items with k cycles.

$$c(n,k) = c(n-1,k-1) + (n-1)c(n-1,k),\ c(0,0) = 1$$
$$\sum_{k=0}^{n} c(n,k)x^k = x(x+1)\dots(x+n-1)$$

$c(8,k) = 0, 5040, 13068, 13132, 6769, 1960, 322, 28, 1$
$c(n,2) = 0, 0, 1, 3, 11, 50, 274, 1764, 13068, 109584, \dots$

Eulerian numbers

Number of permutations of n elements with exactly k descents (equivalently, k ascents or k excedances).

$$E(n,k) = (n-k)E(n-1,k-1) + (k+1)E(n-1,k)$$
$$E(n,0) = E(n,n-1) = 1$$
$$E(n,k) = \sum_{j=0}^{k} (-1)^j \binom{n+1}{j} (k+1-j)^n$$

Stirling numbers of the second kind

Partitions of n distinct elements into exactly k groups.

$$S(n,k) = S(n-1,k-1) + kS(n-1,k)$$
$$S(n,1) = S(n,n) = 1$$
$$S(n,k) = \frac{1}{k!} \sum_{j=0}^{k} (-1)^{k-j} \binom{k}{j} j^n$$

Bell numbers

Total number of partitions of n distinct elements. $B(n) = 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, \dots$. For p prime,

$$B(p^m + n) \equiv mB(n) + B(n+1) \pmod{p}$$

2.3 Number Theory

2.3.1 Maximum number of divisors

Maximum $\tau(n)$ for $n \le N$; last column gives one such n.

N	τ	n
10^2	12	60
10^3	32	840
10^4	64	7560
10^5	128	83160
10^6	240	720720
10^7	448	8648640
10^8	768	73513440
10^9	1344	735134400

N	τ	n
10^{10}	2304	6983776800
10^{11}	4032	97772875200
10^{12}	6720	963761198400
10^{13}	10752	9316358251200
10^{14}	17280	97821761637600
10^{15}	26880	866421317361600
10^{16}	41472	8086598962041600
10^{17}	64512	74801040398884800
10^{18}	103680	897612484786617600

2.4 Graphs

2.4.1 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \ge \dots \ge d_n$ exists iff $d_1 + \dots + d_n$ is even and for every $k = 1 \dots n$,

$$\sum_{i=1}^{k} d_i \le k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k).$$

2.4.2 Maximum Independent Set

Just find a max clique of the complement. If the graph is bipartite, see MinimumVertexCover.h.

2.4.3 Min Cut

After running a max flow, the left side of a min-cut from s to t is given by all vertices reachable from s, only traversing edges with positive residual capacity.

Math (3)

3.1 Number Theory

3.1.1 Modular Arithmetic

ModPow.h b83e45, 7L, 1m30s

```
const ll mod = 1000000007; // faster if const
ll modpow(ll b, ll e) {
  ll ans = 1;
  for (; e; b = b * b % mod, e /= 2)
    if (e & 1) ans = ans * b % mod;
  return ans;
} // d1ec54
```

ModMulLL.h

Description: Calculate $a \cdot b \bmod c$ (or $a^b \bmod c$) for $0 \le a, b \le c \le 7.2 \cdot 10^{18}$.
Time: $\mathcal{O}(1)$ for `modmul`, $\mathcal{O}(\log b)$ for `modpow` bbbd8f, 11L, 2m40s

```
typedef unsigned long long ull;
ull modmul(ull a, ull b, ull M) {
  ll ret = a * b - M * ull(1.L / M * a * b);
  return ret + M * (ret < 0) - M * (ret >= (ll)M);
} // e9309c
ull modpow(ull b, ull e, ull mod) {
  ull ans = 1;
  for (; e; b = modmul(b, b, mod), e /= 2)
    if (e & 1) ans = modmul(ans, b, mod);
  return ans;
} // 100b91
```

ModLog.h

Description: Returns the smallest $x > 0$ s.t. $a^x = b \pmod{m}$, or -1 if no such x exists. modLog(a,1,m) can be used to calculate the order of a.
Memory: $\mathcal{O}(\sqrt{m})$
Time: $\mathcal{O}(\sqrt{m})$ c040b8, 11L, 2m50s

```
ll modLog(ll a, ll b, ll m) {
  ll n = (ll) sqrt(m) + 1, e = 1, f = 1, j = 1;
  unordered_map<ll, ll> A;
  while (j <= n && (e = f = e * a % m) != b % m)
    A[e * b % m] = j++;
  if (e == b % m) return j;
  if (__gcd(m, e) == __gcd(m, b))
    rep(i,2,n+2) if (A.count(e = e * f % m))
      return n * i - A[e];
  return -1;
} // c040b8
```

ModSqrt.h

Description: Tonelli-Shanks algorithm for modular square roots. Finds x s.t. $x^2 = a \pmod{p}$ ($-x$ gives the other solution).
Time: $\mathcal{O}(\log^2 p)$ worst case, $\mathcal{O}(\log p)$ for most p
"ModPow.h" 45c733, 22L, 6m10s

```
const ll p = mod; // mod is p
ll modSqrt(ll a) {
  if(p==2)return a&1;
  a %= p; if (a < 0) a += p;
  if (a == 0) return 0;
  assert(modpow(a, (p-1)/2) == 1); // else no solution
  if (p % 4 == 3) return modpow(a, (p+1)/4);
  // a^(n+3)/8 or 2^(n+3)/8 * 2^(n-1)/4 works if p % 8 == 5
  ll s = p - 1, n = 2;
  int r = 0, m;
  while (s % 2 == 0) ++r, s /= 2;
  while (modpow(n, (p - 1) / 2) != p - 1) ++n;
  ll x = modpow(a, (s + 1) / 2);
  ll b = modpow(a, s), g = modpow(n, s);
  for (;; r = m) {
    ll t = b;
    for (m = 0; m < r && t != 1; ++m) t = t * t % p;
    if (m == 0) return x;
    ll gs = modpow(g, 1LL << (r - m - 1));
    g = gs * gs % p; x = x * gs % p; b = b * g % p;
  } // d569f9
} // d2ac0b
```

CRT.h

Description: Chinese Remainder Theorem.
`crt(a, m, b, n)` computes x such that $x \equiv a \pmod{m}$, $x \equiv b \pmod{n}$. If $|a| < m$ and $|b| < n$, x will obey $0 \le x < \text{lcm}(m,n)$. Assumes $mn < 2^{62}$.
Time: $\log(n)$
"euclid.h" 04d93a, 7L, 2m

```
ll crt(ll a, ll m, ll b, ll n) {
  if (n > m) swap(a, b), swap(m, n);
  ll x, y, g = euclid(m, n, x, y);
  assert((a - b) % g == 0); // else no solution
  x = (b - a) % n * x % n / g * m + a;
  return x < 0 ? x + m*n/g : x;
} // 04d93a
```

3.1.2 Primality and Divisibility

FastEratosthenes.h

Description: Prime sieve for generating all primes smaller than LIM.
Time: LIM=1e9 $\approx$ 1.5s 6b2912, 20L, 5m40s

```
const int LIM = 1e6;
bitset<LIM> isPrime;
vi eratosthenes() {
  const int S = (int)round(sqrt(LIM)), R = LIM / 2;
  vi pr = {2}, sieve(S+1); pr.reserve(int(LIM/log(LIM)*1.1));
  vector<pii> cp;
  for (int i = 3; i <= S; i += 2) if (!sieve[i]) {
    cp.push_back({i, i * i / 2});
    for (int j = i * i; j <= S; j += 2 * i) sieve[j] = 1;
  } // d22e52
  for (int L = 1; L <= R; L += S) {
    array<bool, S> block{};
    for (auto &[p, idx] : cp)
      for (int i=idx; i < S+L; idx = (i+=p)) block[i-L] = 1;
    rep(i,0,min(S, R - L))
```

```
      if (!block[i]) pr.push_back((L + i) * 2 + 1);
  } // 5b6623
  for (int i : pr) isPrime[i] = 1;
  return pr;
} // 8ee6d2
```

MillerRab.h

Description: Deterministic Miller-Rabin primality test. Guaranteed to work for numbers up to $7 \cdot 10^{18}$; for larger numbers, use Python and extend A randomly.

Time: 7 times the complexity of $a^b \mod c$.

"ModMulLL.h" 60dcd1, 12L, 3m30s

```
bool isPrime(ull n) {
  if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
  ull A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
      s = __builtin_ctzll(n-1), d = n >> s;
  for (ull a : A) {   // ^ count trailing zeroes
    ull p = modpow(a%n, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && i--)
      p = modmul(p, p, n);
    if (p != n-1 && i != s) return 0;
  } // edfaf1
  return 1;
} // 60dcd1
```

PollardRho.h

Description: Pollard-rho randomized factorization algorithm. Returns prime factors of a number, in arbitrary order (e.g. 2299 -> {11, 19, 11}).

Time: $\mathcal{O}\left(n^{1/4}\right)$, less for numbers with small factors.

"ModMulLL.h", "MillerRabin.h" d8d98d, 18L, 4m30s

```
ull pollard(ull n) {
  ull x = 0, y = 0, t = 30, prd = 2, i = 1, q;
  auto f = [&](ull x) { return modmul(x, x, n) + i; };
  while (t++ % 40 || __gcd(prd, n) == 1) {
    if (x == y) x = ++i, y = f(x);
    if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
    x = f(x), y = f(f(y));
  } // 989d40
  return __gcd(prd, n);
} // cd2ac3
vector<ull> factor(ull n) {
  if (n == 1) return {};
  if (isPrime(n)) return {n};
  ull x = pollard(n);
  auto l = factor(x), r = factor(n / x);
  l.insert(l.end(), all(r));
  return l;
} // d54ba8
```

euclid.h

Description: Finds two integers x and y, such that $ax + by = \gcd(a, b)$. If you just need gcd, use the built in __gcd instead. If a and b are coprime, then x is the inverse of $a \pmod b$.

33ba8f, 5L, 1m10s

```
ll euclid(ll a, ll b, ll &x, ll &y) {
  if (!b) return x = 1, y = 0, a;
  ll d = euclid(b, a % b, y, x);
  return y -= a/b * x, d;
} // 33ba8f
```

3.2 Discrete Math

3.2.1 Convolutions

FastFourierTransform.h

Description: fft(a) computes $f(k) = \sum_x a[x] \exp(2\pi i \cdot kx/N)$ for all k. N must be a power of 2. Useful for convolution: conv(a, b) = c, where $c[x] = \sum a[i]b[x-i]$. For convolution of complex numbers or more than two vectors: FFT, multiply pointwise, divide by n, reverse(start+1, end), FFT back. Rounding is safe if $(\sum a_i^2 + \sum b_i^2) \log_2 N < 9 \cdot 10^{14}$ (in practice 10^{16}; higher for random inputs). Otherwise, use NTT/FFTMod.

Time: $\mathcal{O}(N \log N)$ with $N = |A| + |B|$ ($\sim 1s$ for $N = 2^{22}$)

3dd197, 36L, 11m10s

```
typedef complex<double> C;
typedef vector<double> vd;
void fft(vector<C>& a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vector<complex<long double>> R(2, 1); // (10% faster if double)
  static vector<C> rt(2, 1);
  for (static int k = 2; k < n; k *= 2) {
    R.resize(n); rt.resize(n);
    auto x = polar(1.0L, acos(-1.0L) / k);
    rep(i,k,2*k) rt[i] = R[i] = i&1 ? R[i/2] * x : R[i/2];
  } // a8a74e
  vi rev(n);
  rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
      auto x = (double *)&rt[j+k], y = (double *)&a[i+j+k];
      C z(x[0]*y[0] - x[1]*y[1], x[0]*y[1] + x[1]*y[0]);
      a[i + j + k] = a[i + j] - z;
      a[i + j] += z;
    } // 69f11b
} // 2258f7
vd conv(const vd& a, const vd& b) {
  if (a.empty() || b.empty()) return {};
  vd res(sz(a) + sz(b) - 1);
  int L = 32 - __builtin_clz(sz(res)), n = 1 << L;
  vector<C> in(n), out(n);
  copy(all(a), begin(in));
  rep(i,0,sz(b)) in[i].imag(b[i]);
  fft(in);
  for (C& x : in) x *= x;
  rep(i,0,n) out[i] = in[-i & (n - 1)] - conj(in[i]);
  fft(out);
  rep(i,0,sz(res)) res[i] = imag(out[i]) / (4 * n);
  return res;
} // 873509
```

FastFourierTransformMod.h

Description: Higher precision FFT, can be used for convolutions modulo arbitrary integers as long as $N \log_2 N \cdot \text{mod} < 8.6 \cdot 10^{14}$ (in practice 10^{16} or higher). Inputs must be in $[0, \text{mod})$.

Time: $\mathcal{O}(N \log N)$, where $N = |A| + |B|$ (twice as slow as NTT or FFT)

"FastFourierTransform.h" b82773, 22L, 7m10s

```
typedef vector<ll> vl;
template<int M> vl convMod(const vl &a, const vl &b) {
  if (a.empty() || b.empty()) return {};
  vl res(sz(a) + sz(b) - 1);
  int B=32-__builtin_clz(sz(res)), n=1<<B, cut=int(sqrt(M));
  vector<C> L(n), R(n), outs(n), outl(n);
  rep(i,0,sz(a)) L[i] = C((int)a[i] / cut, (int)a[i] % cut);
  rep(i,0,sz(b)) R[i] = C((int)b[i] / cut, (int)b[i] % cut);
  fft(L), fft(R);
  rep(i,0,n) {
    int j = -i & (n - 1);
    outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
    outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
  } // cb3017
  fft(outl), fft(outs);
  rep(i,0,sz(res)) {
    ll av = ll(real(outl[i])+.5), cv = ll(imag(outs[i])+.5);
    ll bv = ll(imag(outl[i])+.5) + ll(real(outs[i])+.5);
    res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
  } // 58fa4f
  return res;
} // c1f2f1
```

NumberTheoreticTransform.h

Description: ntt(a) computes $f(k) = \sum_x a[x] g^{xk}$ for all k, where $g = \text{root}^{(mod-1)/N}$. N must be a power of 2. Useful for convolution modulo specific nice primes of the form $2^a b + 1$, where the convolution result has size at most 2^a. For arbitrary modulo, see FFTMod. conv(a, b) = c, where $c[x] = \sum a[i]b[x-i]$. For manual convolution: NTT the inputs, multiply pointwise, divide by n, reverse(start+1, end), NTT back. Inputs must be in $[0, \text{mod})$.

Time: $\mathcal{O}(N \log N)$

"../math/ModPow.h" a46443, 35L, 11m20s

```
const ll root = 62; // change mod to (119 << 23) + 1 = 998244353
// For p < 2^30 there is also e.g. 5 << 25, 7 << 26, 479 << 21
// and 483 << 21 (same root). The last two are > 10^9.
typedef vector<ll> vl;
void ntt(vl &a) {
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vl rt(2, 1);
  for (static int k = 2, s = 2; k < n; k *= 2, s++) {
    rt.resize(n);
    ll z[] = {1, modpow(root, mod >> s)};
    rep(i,k,2*k) rt[i] = rt[i / 2] * z[i & 1] % mod;
  } // f39127
  vi rev(n);
  rep(i,0,n) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  rep(i,0,n) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k) rep(j,0,k) {
      ll z = rt[j + k] * a[i + j + k] % mod, &ai = a[i + j];
      a[i + j + k] = ai - z + (z > ai ? mod : 0);
      ai += (ai + z >= mod ? z - mod : z);
    } // 9a8565
} // 3b763b
vl conv(const vl &a, const vl &b) {
  if (a.empty() || b.empty()) return {};
  int s = sz(a) + sz(b) - 1, B = 32 - __builtin_clz(s),
      n = 1 << B;
  int inv = modpow(n, mod - 2);
  vl L(a), R(b), out(n);
  L.resize(n), R.resize(n);
  ntt(L), ntt(R);
  rep(i,0,n)
    out[-i & (n - 1)] = (ll)L[i] * R[i] % mod * inv % mod;
  ntt(out);
  return {out.begin(), out.begin() + s};
} // 3876bf
```

FastSubsetTransform.h

Description: Transform to a basis with fast convolutions of the form $c[z] = \sum_{z=x \oplus y} a[x] \cdot b[y]$, where $\oplus$ is one of AND, OR, XOR. The size of a must be a power of two, and a and b must have the same power-of-two size.

Time: $\mathcal{O}(N \log N)$

464cf3, 16L, 4m30s

```
void FST(vi& a, bool inv) {
  for (int n = sz(a), step = 1; step < n; step *= 2) {
    for (int i = 0; i < n; i += 2 * step) rep(j,i,i+step) {
      int &u = a[j], &v = a[j + step]; tie(u, v) =
        inv ? pii(v - u, u) : pii(v, u + v); // AND
        inv ? pii(v, u - v) : pii(u + v, u); // OR
```

```
      pii(u + v, u - v);                    // XOR
    } // 7b7e71
  } // faec61
  if (inv) for (int& x : a) x /= sz(a); // XOR only
} // 57eeaf
vi conv(vi a, vi b) {
  FST(a, 0); FST(b, 0);
  rep(i,0,sz(a)) a[i] *= b[i];
  FST(a, 1); return a;
} // 3cbd18
```

3.2.2 Recurrences and Sums

BerlekampMassey.h

Description: Recovers any n-order linear recurrence relation from the first $2n$ terms of the recurrence. Useful for guessing linear recurrences after brute-forcing the first terms. The idea should generalize to any field, but numerical stability for floats is not guaranteed. Output will have size $\leq n$.
Usage: `berlekampMassey({0, 1, 1, 3, 5, 11}) // {1, 2}`
Time: $\mathcal{O}\left(N^2\right)$

"../math/ModPow.h" 96548b, 17L, 4m30s

```
vector<ll> berlekampMassey(vector<ll> s) {
  int n = sz(s), L = 0, m = 0;
  vector<ll> C(n), B(n), T;
  C[0] = B[0] = 1; ll b = 1;
  rep(i,0,n) { ++m;
    ll d = s[i] % mod;
    rep(j,1,L+1) d = (d + C[j] * s[i - j]) % mod;
    if (!d) continue;
    T = C; ll coef = d * modpow(b, mod-2) % mod;
    rep(j,m,n) C[j] = (C[j] - coef * B[j - m]) % mod;
    if (2 * L > i) continue;
    L = i + 1 - L; B = T; b = d; m = 0;
  } // 8c2376
  C.resize(L + 1); C.erase(C.begin());
  for (ll& x : C) x = (mod - x) % mod;
  return C;
} // 96548b
```

LinearRecurrence.h

Description: Generates the k'th term of an n-order linear recurrence $S[i] = \sum_j S[i-j-1]tr[j]$, given $S[0 \ldots \geq n-1]$ and $tr[0 \ldots n-1]$. Faster than matrix multiplication. Useful together with Berlekamp–Massey.
Usage: `linearRec({0, 1}, {1, 1}, k) // k'th Fibonacci number`
Time: $\mathcal{O}\left(n^2 \log k\right)$

f4e444, 21L, 5m20s

```
typedef vector<ll> Poly;
ll linearRec(Poly S, Poly tr, ll k) {
  int n = sz(tr);
  auto combine = [&](Poly a, Poly b) {
    Poly res(n * 2 + 1);
    rep(i,0,n+1) rep(j,0,n+1)
      res[i + j] = (res[i + j] + a[i] * b[j]) % mod;
    for (int i = 2 * n; i > n; --i) rep(j,0,n)
      res[i - 1 - j] = (res[i - 1 - j] + res[i] * tr[j]) % mod;
    res.resize(n + 1);
    return res;
  }; // 55c8ab
  Poly pol(n + 1), e(pol); pol[0] = e[1] = 1;
  for (++k; k; k /= 2) {
    if (k % 2) pol = combine(pol, e);
    e = combine(e, e);
  } // 8137be
  ll res = 0;
  rep(i,0,n) res = (res + pol[i + 1] * S[i]) % mod;
  return res;
} // 5948dc
```

FloorModSum.h

Description: Floor sum and Mod sum.
`modsum(to, c, k, m)` $= \sum_{i=0}^{to-1} (ki+c)\%m$. `divsum` is similar but for floored division.
Time: $\log(m)$, with a large constant.

5c5bc5, 15L, 4m40s

```
typedef unsigned long long ull;
// written in a weird way to deal with overflows correctly
ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }
ull divsum(ull to, ull c, ull k, ull m) {
  ull res = k / m * sumsq(to) + c / m * to;
  k %= m; c %= m;
  if (!k) return res;
  ull to2 = (to * k + c) / m;
  return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
} // 78bfc8
ll modsum(ull to, ll c, ll k, ll m) {
  c = ((c % m) + m) % m;
  k = ((k % m) + m) % m;
  return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
} // 5daf3e
```

3.3 Linear Algebra

3.3.1 Vector Basis

XorBasis.h

Description: Linear basis over xor. add(x) inserts x and returns whether it was independent. can(x) checks if x can be formed. maxXor(x) returns the maximum value obtainable by xoring x with a subset of the basis.
Memory: $\mathcal{O}(B)$.
Time: $\mathcal{O}(B)$ per operation.

0b5873, 19L, 3m

```
struct XorBasis {
  static constexpr int B=63;
  array<ll,B> b{};
  bool add(ll x){
    for(int i=B;i--;)if(x>>i&1){
      if(!b[i])return b[i]=x,1;
      x^=b[i];
    } // 813efa
    return 0;
  } // 518ac0
  bool can(ll x) const {
    for(int i=B;i--;)if(x>>i&1)x^=b[i];
    return !x;
  } // 97bd4f
  ll maxXor(ll x=0) const {
    for(int i=B;i--;)x=max(x,x^b[i]);
    return x;
  } // e67fdb
}; // 0b5873
```

3.3.2 Matrices

Determinant.h

Description: Calculates determinant of a matrix. Destroys the matrix.
Time: $\mathcal{O}\left(N^3\right)$

bd5cec, 15L, 3m20s

```
double det(vector<vector<double>>& a) {
  int n = sz(a); double res = 1;
  rep(i,0,n) {
    int b = i;
    rep(j,i+1,n) if (fabs(a[j][i]) > fabs(a[b][i])) b = j;
    if (i != b) swap(a[i], a[b]), res *= -1;
    res *= a[i][i];
    if (res == 0) return 0;
    rep(j,i+1,n) {
      double v = a[j][i] / a[i][i];
      if (v != 0) rep(k,i+1,n) a[j][k] -= v * a[i][k];
    } // 4ec6a2
  } // ee1466
  return res;
} // bd5cec
```

IntDeterminant.h

Description: Calculates determinant using modular arithmetics. Modulos can also be removed to get a pure-integer version.
Time: $\mathcal{O}\left(N^3 \log mod\right)$

3e3b26, 17L, 3m20s

```
ll det(vector<vector<ll>>& a, ll mod) {
  int n = sz(a); ll ans = 1;
  rep(i,0,n) {
    rep(j,i+1,n) {
      while (a[j][i] != 0) { // gcd step
        ll t = a[i][i] / a[j][i];
        if (t) rep(k,i,n)
          a[i][k] = (a[i][k] - a[j][k] * t) % mod;
        swap(a[i], a[j]);
        ans *= -1;
      } // e81b29
    } // 30d1b2
    ans = ans * a[i][i] % mod;
    if (!ans) return 0;
  } // f39a45
  return (ans + mod) % mod;
} // 3e3b26
```

MatrixInverse.h

Description: Invert matrix A. Returns rank; result is stored in A unless singular (rank < n). Can easily be extended to prime moduli; for prime powers, repeatedly set $A^{-1} = A^{-1}(2I - AA^{-1}) \pmod{p^k}$ where A^{-1} starts as the inverse of A mod p, and k is doubled in each step.
Time: $\mathcal{O}\left(n^3\right)$

ebfff6, 32L, 8m10s

```
int matInv(vector<vector<double>>& A) {
  int n = sz(A); vi col(n);
  vector<vector<double>> tmp(n, vector<double>(n));
  rep(i,0,n) tmp[i][i] = 1, col[i] = i;
  rep(i,0,n) {
    int r = i, c = i;
    rep(j,i,n) rep(k,i,n)
      if (fabs(A[j][k]) > fabs(A[r][c]))
        r = j, c = k;
    if (fabs(A[r][c]) < 1e-12) return i;
    A[i].swap(A[r]); tmp[i].swap(tmp[r]);
    rep(j,0,n)
      swap(A[j][i], A[j][c]), swap(tmp[j][i], tmp[j][c]);
    swap(col[i], col[c]);
    double v = A[i][i];
    rep(j,i+1,n) {
      double f = A[j][i] / v;
      A[j][i] = 0;
      rep(k,i+1,n) A[j][k] -= f*A[i][k];
      rep(k,0,n) tmp[j][k] -= f*tmp[i][k];
    } // ebeea3
    rep(j,i+1,n) A[i][j] /= v;
    rep(j,0,n) tmp[i][j] /= v;
    A[i][i] = 1;
  } // 26d90b
  for (int i = n-1; i > 0; --i) rep(j,0,i) {
    double v = A[j][i];
    rep(k,0,n) tmp[j][k] -= v*tmp[i][k];
  } // 03ae0c
  rep(i,0,n) rep(j,0,n) A[col[i]][col[j]] = tmp[i][j];
  return n;
} // ebfff6
```

3.3.3 Systems

SolveLinear.h

Description: Solves $A * x = b$. If there are multiple solutions, an arbitrary one is returned. Returns rank, or -1 if no solutions. Data in A and b is lost.
Time: $\mathcal{O}(n^2 m)$

44c9ab, 46L, 11m

```
typedef vector<double> vd;
const double eps = 1e-12;
int solveLinear(vector<vd>& A, vd& b, vd& x) {
  int n = sz(A), m = sz(x), rank = 0, br, bc;
  if (n) assert(sz(A[0]) == m);
  vi col(m); iota(all(col), 0);
  rep(i,0,n) {
    double v, bv = 0;
    rep(r,i,n) rep(c,i,m)
      if ((v = fabs(A[r][c])) > bv)
        br = r, bc = c, bv = v;
    if (bv <= eps) {
      rep(j,i,n) if (fabs(b[j]) > eps) return -1;
      break;
    } // c92205
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j,0,n) swap(A[j][i], A[j][bc]);
    bv = 1/A[i][i];
    rep(j,i+1,n) {
      double fac = A[j][i] * bv;
      b[j] -= fac * b[i];
      rep(k,i+1,m) A[j][k] -= fac*A[i][k];
    } // 881860
    rank++;
  } // 0f0f0f
  x.assign(m, 0);
  for (int i = rank; i--;) {
    b[i] /= A[i][i];
    x[col[i]] = b[i];
    rep(j,0,i) b[j] -= A[j][i] * b[i];
  } // ed1d08
  return rank; // (multiple solutions if rank < m)
} // ade67b

/*
To get all uniquely determined values of $x$ back from
     SolveLinear, make the following changes:
rep(j,0,n) if (j != i) // instead of rep(j,i+1,n)
// ... then at the end:
x.assign(m, undefined);
rep(i,0,rank) {
  rep(j,rank,m) if (fabs(A[i][j]) > eps) goto fail;
  x[col[i]] = b[i] / A[i][i];
fail:; } // 87878c
*/
```

SolveLinearBinary.h

Description: Solves $Ax = b$ over $\mathbb{F}_2$. If there are multiple solutions, one is returned arbitrarily. Returns rank, or -1 if no solutions. Destroys A and b.
Time: $\mathcal{O}(n^2 m)$

fa2d7a, 32L, 6m40s

```
typedef bitset<1000> bs;
int solveLinear(vector<bs>& A, vi& b, bs& x, int m) {
  int n = sz(A), rank = 0, br;
  assert(m <= sz(x));
  vi col(m); iota(all(col), 0);
  rep(i,0,n) {
    for (br=i; br<n; ++br) if (A[br].any()) break;
    if (br == n) {
      rep(j,i,n) if(b[j]) return -1;
      break;
    } // 80687c
    int bc = (int)A[br]._Find_next(i-1);
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    rep(j,0,n) if (A[j][i] != A[j][bc]) {
      A[j].flip(i); A[j].flip(bc);
    } // b44a9b
    rep(j,i+1,n) if (A[j][i]) {
      b[j] ^= b[i];
      A[j] ^= A[i];
    } // 87192e
    rank++;
  } // fe9281
  x = bs();
  for (int i = rank; i--;) {
    if (!b[i]) continue;
    x[col[i]] = 1;
    rep(j,0,i) b[j] ^= A[j][i];
  } // 8fdba1
  return rank; // (multiple solutions if rank < m)
} // 26d73e
```

Tridiagonal.h

Description: $x = \text{tridiagonal}(d, p, q, b)$ solves the equation system

$$\begin{pmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_{n-1} \end{pmatrix} = \begin{pmatrix} d_0 & p_0 & 0 & 0 & \cdots & 0 \\ q_0 & d_1 & p_1 & 0 & \cdots & 0 \\ 0 & q_1 & d_2 & p_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & q_{n-3} & d_{n-2} & p_{n-2} \\ 0 & 0 & \cdots & 0 & q_{n-2} & d_{n-1} \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ \vdots \\ x_{n-1} \end{pmatrix}.$$

This is useful for solving problems on the type

$$a_i = b_i a_{i-1} + c_i a_{i+1} + d_i,\ 1 \le i \le n,$$

where a_0, a_{n+1}, b_i, c_i and d_i are known. a can then be obtained from

$$\{a_i\} = \text{tridiagonal}(\{1, -1, -1, ..., -1, 1\}, \{0, c_1, c_2, \ldots, c_n\}, \{b_1, b_2, \ldots, b_n, 0\}, \{a_0, d_1, d_2, \ldots, d_n, a_{n+1}\}).$$

Fails if the solution is not unique.
If $|d_i| > |p_i| + |q_{i-1}|$ for all i, or $|d_i| > |p_{i-1}| + |q_i|$, or the matrix is positive definite, the algorithm is numerically stable and neither `tr` nor the check for `diag[i] == 0` is needed.
Time: $\mathcal{O}(N)$

8f9fa8, 26L, 6m

```
typedef double T;
vector<T> tridiagonal(vector<T> diag, const vector<T>& super,
    const vector<T>& sub, vector<T> b) {
  int n = sz(b); vi tr(n);
  rep(i,0,n-1) {
    if (abs(diag[i]) < 1e-9 * abs(super[i])) { // diag[i] == 0
      b[i+1] -= b[i] * diag[i+1] / super[i];
      if (i+2 < n) b[i+2] -= b[i] * sub[i+1] / super[i];
      diag[i+1] = sub[i]; tr[++i] = 1;
    } else { // 464c09
      diag[i+1] -= super[i]*sub[i]/diag[i];
      b[i+1] -= b[i]*sub[i]/diag[i];
    } // 62de5a
  } // ed9cce
  for (int i = n; i--;) {
    if (tr[i]) {
      swap(b[i], b[i-1]);
      diag[i-1] = diag[i];
      b[i] /= super[i-1];
    } else { // 0543e4
      b[i] /= diag[i];
      if (i) b[i-1] -= b[i]*super[i-1];
    } // aa91c6
  } // 28af28
  return b;
} // 06d549
```

3.3.4 Optimization

Simplex.h

Description: Solves a general linear maximization problem: maximize $c^T x$ subject to $Ax \le b$, $x \ge 0$. Returns -inf if there is no solution, inf if there are arbitrarily good solutions, or the maximum value of $c^T x$ otherwise. The input vector is set to an optimal x (or in the unbounded case, an arbitrary solution fulfilling the constraints). Numerical stability is not guaranteed. For better performance, define variables such that $x = 0$ is viable.
Usage: `vvd A = {{1,-1}, {-1,1}, {-1,-2}};`
`vd b = {1,1,-4}, c = {-1,-1}, x;`
`T val = LPSolver(A, b, c).solve(x);`
Time: $\mathcal{O}(NM * \#pivots)$, where a pivot may be e.g. an edge relaxation. $\mathcal{O}(2^n)$ in the general case.

aa8530, 60L, 16m50s

```
typedef double T; // long double, Rational, double + mod<P>...
typedef vector<T> vd;
typedef vector<vd> vvd;
const T eps = 1e-8, inf = 1/.0;
#define MP make_pair
#define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s])) s=j
struct LPSolver {
  int m, n; vi N, B; vvd D;
  LPSolver(const vvd& A, const vd& b, const vd& c) :
    m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
      rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];
      rep(i,0,m) { B[i] = n+i; D[i][n] = -1; D[i][n+1] = b[i];}
      rep(j,0,n) { N[j] = j; D[m][j] = -c[j]; }
      N[n] = -1; D[m+1][n] = 1;
    } // 6ff8e9
  void pivot(int r, int s) {
    T *a = D[r].data(), inv = 1 / a[s];
    rep(i,0,m+2) if (i != r && abs(D[i][s]) > eps) {
      T *b = D[i].data(), inv2 = b[s] * inv;
      rep(j,0,n+2) b[j] -= a[j] * inv2;
      b[s] = a[s] * inv2;
    } // ca4460
    rep(j,0,n+2) if (j != s) D[r][j] *= inv;
    rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
    D[r][s] = inv;
    swap(B[r], N[s]);
  } // 9cd0a8
  bool simplex(int phase) {
    int x = m + phase - 1;
    for (;;) {
      int s = -1;
      rep(j,0,n+1) if (N[j] != -phase) ltj(D[x]);
      if (D[x][s] >= -eps) return true;
      int r = -1;
      rep(i,0,m) {
        if (D[i][s] <= eps) continue;
        if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
                     < MP(D[r][n+1] / D[r][s], B[r])) r = i;
      } // 46853f
      if (r == -1) return false;
      pivot(r, s);
    } // 7d839b
  } // f15644
  T solve(vd &x) {
    int r = 0;
    rep(i,1,m) if (D[i][n+1] < D[r][n+1]) r = i;
    if (D[r][n+1] < -eps) {
      pivot(r, n);
      if (!simplex(2) || D[m+1][n+1] < -eps) return -inf;
      rep(i,0,m) if (B[i] == -1) {
        int s = 0;
```

```
      rep(j,1,n+1) ltj(D[i]);
      pivot(i, s);
    } // 683310
  } // b6553f
  bool ok = simplex(1); x = vd(n);
  rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n+1];
  return ok ? D[m][n+1] : inf;
 } // 396a95
}; // c57b35
```

3.4 Numerical Calculus

3.4.1 Functions

Integrate.h

Description: Simple integration of a function over an interval using Simpson's rule. The error should be proportional to h^4, although in practice you will want to verify that the result is stable to desired precision when epsilon changes.

4756fc, 6L, 1m40s

```
template<class F>
double quad(double a, double b, F f, const int n = 1000) {
  double h = (b - a) / 2 / n, v = f(a) + f(b);
  rep(i,1,n*2) v += f(a + i*h) * (i&1 ? 4 : 2);
  return v * h / 3;
} // ddcce2
```

IntegrateAdaptive.h

Description: Fast integration using an adaptive Simpson's rule.

Usage:
```
double sphereVolume = quad(-1, 1, [](double x) {
return quad(-1, 1, [&](double y) {
return quad(-1, 1, [&](double z) {
return x*x + y*y + z*z < 1; });});});
```

92dd79, 12L, 3m50s

```
typedef double d;
#define S(a,b) (f(a) + 4*f((a+b) / 2) + f(b)) * (b-a) / 6
template <class F> d rec(F& f, d a, d b, d eps, d S) {
  d c = (a + b) / 2;
  d S1 = S(a, c), S2 = S(c, b), T = S1 + S2;
  if (abs(T - S) <= 15 * eps || b - a < 1e-10)
    return T + (T - S) / 15;
  return rec(f, a, c, eps / 2, S1) + rec(f, c, b, eps / 2, S2);
} // 10c680
template<class F> d quad(d a, d b, F f, d eps = 1e-8) {
  return rec(f, a, b, eps, S(a, b));
} // 7afc73
```

3.4.2 Polynomials

PolyRoots.h

Description: Finds the real roots to a polynomial. xmin and xmax should contain all real roots. Multiple roots may produce duplicate nearby answers; deduplicate if necessary.

Usage: `polyRoots({{2,-3,1}},-1e9,1e9) // solve x^2-3x+2 = 0`

Time: $\mathcal{O}\left(n^2 \log(1/\epsilon)\right)$

e897c3, 40L, 8m50s

```
struct Poly {
  vector<double> a;
  double operator()(double x) const {
    double val = 0;
    for (int i = sz(a); i--;) (val *= x) += a[i];
    return val;
  } // ae76f3
  void diff() {
    rep(i,1,sz(a)) a[i-1] = i*a[i];
    a.pop_back();
  } // afcaea
  void divroot(double x0) {
    double b = a.back(), c; a.back() = 0;
    for(int i=sz(a)-1; i--;) c = a[i], a[i] = a[i+1]*x0+b, b=c;
    a.pop_back();
  } // 3f874a
}; // c9b7b0
vector<double> polyRoots(Poly p, double xmin, double xmax) {
  if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }
  vector<double> ret;
  Poly der = p;
  der.diff();
  auto dr = polyRoots(der, xmin, xmax);
  dr.push_back(xmin-1);
  dr.push_back(xmax+1);
  sort(all(dr));
  rep(i,0,sz(dr)-1) {
    double l = dr[i], h = dr[i+1];
    bool sign = p(l) > 0;
    if (sign ^ (p(h) > 0)) {
      rep(it,0,60) { // while (h - l > 1e-8)
        double m = (l + h) / 2, f = p(m);
        if ((f <= 0) ^ sign) l = m;
        else h = m;
      } // b69f41
      ret.push_back((l + h) / 2);
    } // fc22f0
  } // d15986
  return ret;
} // b00bfe
```

PolyInterpolate.h

Description: Given n points (x[i], y[i]), computes an n-1-degree polynomial p that passes through them: $p(x) = a[0] * x^0 + ... + a[n-1] * x^{n-1}$. For numerical precision, pick $x[k] = c * \cos(k/(n-1) * \pi), k = 0 \ldots n-1$.

Time: $\mathcal{O}\left(n^2\right)$

08bf48, 13L, 2m40s

```
typedef vector<double> vd;
vd interpolate(vd x, vd y, int n) {
  vd res(n), temp(n);
  rep(k,0,n-1) rep(i,k+1,n)
    y[i] = (y[i] - y[k]) / (x[i] - x[k]);
  double last = 0; temp[0] = 1;
  rep(k,0,n) rep(i,0,n) {
    res[i] += y[k] * temp[i];
    swap(last, temp[i]);
    temp[i] -= last * x[k];
  } // 4c74fe
  return res;
} // 285367
```

Geometry (4)

4.1 Geometric primitives

Point.h

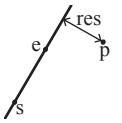
Description: Class to handle points in the plane. T can be e.g. double or long long. (Avoid int.)

ef0c0e, 28L, 12m

```
template <class T> int sgn(T x) { return (x > 0) - (x < 0); }
template<class T>
struct Point {
  typedef Point P;
  T x, y;
  explicit Point(T _x=0, T _y=0) : x(_x), y(_y) {}
  bool operator<(P p) const { return tie(x,y) < tie(p.x,p.y); }
  bool operator==(P p) const { return tie(x,y)==tie(p.x,p.y); }
  P operator+(P p) const { return P(x+p.x, y+p.y); }
  P operator-(P p) const { return P(x-p.x, y-p.y); }
  P operator*(T d) const { return P(x*d, y*d); }
  P operator/(T d) const { return P(x/d, y/d); }
  T dot(P p) const { return x*p.x + y*p.y; }
  T cross(P p) const { return x*p.y - y*p.x; }
  T cross(P a, P b) const { return (a-*this).cross(b-*this); }
  T dist2() const { return x*x + y*y; }
  double dist() const { return sqrt((double)dist2()); }
  // angle to x-axis in interval [-pi, pi]
  double angle() const { return atan2(y, x); }
  P unit() const { return *this/dist(); } // makes dist()=1
  P perp() const { return P(-y, x); } // rotates +90 degrees
  P normal() const { return perp().unit(); }
  // returns point rotated 'a' radians ccw around the origin
  P rotate(double a) const {
    return P(x*cos(a)-y*sin(a),x*sin(a)+y*cos(a)); } // 4822a3
  friend ostream& operator<<(ostream& os, P p) {
    return os << "(" << p.x << "," << p.y << ")"; } // 9a9c95
}; // 6a5cce
```

lineDistance.h

Description:
Returns the signed distance between point p and the line containing points a and b. Positive value on left side and negative on right as seen from a towards b. a==b gives nan. P is supposed to be Point<T> or Point3D<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long. Using Point3D will always give a non-negative distance. For Point3D, call .dist on the result of the cross product.

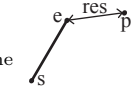


"Point.h" f6bf6b, 4L, 1m

```
template<class P>
double lineDist(const P& a, const P& b, const P& p) {
  return (double)(b-a).cross(p-a)/(b-a).dist();
} // 00891c
```

SegmentDistance.h

Description:
Returns the shortest distance between point p and the line segment from point s to e.



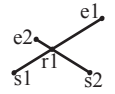
Usage:
```
Point<double> a, b(2,2), p(1,1);
bool onSegment = segDist(a,b,p) < 1e-10;
```

"Point.h" 5c88f4, 6L, 1m40s

```
typedef Point<double> P;
double segDist(P& s, P& e, P& p) {
  if (s==e) return (p-s).dist();
  auto d = (e-s).dist2(), t = min(d,max(.0,(p-s).dot(e-s)));
  return ((p-s)*d-(e-s)*t).dist()/d;
} // ae751a
```

SegmentIntersection.h

Description:
If a unique intersection point between the line segments going from s1 to e1 and from s2 to e2 exists then it is returned. If no intersection point exists an empty vector is returned. If infinitely many exist a vector with 2 elements is returned, containing the endpoints of the common line segment. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or long long.



Usage:
```
vector<P> inter = segInter(s1,e1,s2,e2);
if (sz(inter)==1)
cout << "segments intersect at " << inter[0] << endl;
```

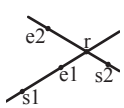
"Point.h", "OnSegment.h" 9d57f2, 13L, 4m30s

```
template<class P> vector<P> segInter(P a, P b, P c, P d) {
  auto oa = c.cross(d, a), ob = c.cross(d, b),
       oc = a.cross(b, c), od = a.cross(b, d);
  // Checks if intersection is single non-endpoint point.
  if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
```

```
    return {(a * ob - b * oa) / (ob - oa)};
  set<P> s;
  if (onSegment(c, d, a)) s.insert(a);
  if (onSegment(c, d, b)) s.insert(b);
  if (onSegment(a, b, c)) s.insert(c);
  if (onSegment(a, b, d)) s.insert(d);
  return {all(s)};
} // 9d57f2
```

lineIntersection.h

Description:
If a unique intersection point of the lines going through s1,e1 and s2,e2 exists {1, point} is returned. If no intersection point exists {0, (0,0)} is returned and if infinitely many exists {-1, (0,0)} is returned. The wrong position will be returned if P is Point<ll> and the intersection point does not have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow if using int or ll.



Usage: `auto res = lineInter(s1,e1,s2,e2);`

```
if (res.first == 1)
cout << "intersection point at " << res.second << endl;
```

"Point.h" a01f81, 8L, 2m20s

```
template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
  auto d = (e1 - s1).cross(e2 - s2);
  if (d == 0) // if parallel
    return {-(s1.cross(e1, s2) == 0), P(0, 0)};
  auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
  return {1, (s1 * p + e1 * q) / d};
} // 47279a
```

sideOf.h

Description: Returns where p is as seen from s towards e. 1/0/-1 $\Leftrightarrow$ left/on line/right. If the optional argument *eps* is given 0 is returned if p is within distance *eps* from the line. P is supposed to be Point<T> where T is e.g. double or long long. It uses products in intermediate steps so watch out for overflow if using int or long long.

Usage: `bool left = sideOf(p1,p2,q)==1;`

"Point.h" 3af81c, 9L, 2m10s

```
template<class P>
int sideOf(P s, P e, P p) { return sgn(s.cross(e, p)); }

template<class P>
int sideOf(const P& s, const P& e, const P& p, double eps) {
  auto a = (e-s).cross(p-s);
  double l = (e-s).dist()*eps;
  return (a > l) - (a < -l);
} // 33fa03
```

OnSegment.h

Description: Returns true iff p lies on the line segment from s to e. Use `(segDist(s,e,p)<=epsilon)` instead when using Point<double>.

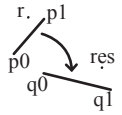
"Point.h" c597e8, 3L, 50s

```
template<class P> bool onSegment(P s, P e, P p) {
  return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
} // c597e8
```

linearTransformation.h

Description:

Apply the linear transformation (translation, rotation and scaling) which takes line p0-p1 to line q0-q1 to point r.



"Point.h" 03a306, 6L, 2m10s

```
typedef Point<double> P;
P linearTransformation(const P& p0, const P& p1,
    const P& q0, const P& q1, const P& r) {
  P dp = p1-p0, dq = q1-q0, num(dp.cross(dq), dp.dot(dq));
  return q0 + P((r-p0).cross(num), (r-p0).dot(num))/dp.dist2();
} // 45ea01
```

LineProjectionReflection.h

Description: Projects point p onto line ab. Set refl=true to get reflection of point p across line ab instead. The wrong point will be returned if P is an integer point and the desired point doesn't have integer coordinates. Products of three coordinates are used in intermediate steps so watch out for overflow.

"Point.h" b5562d, 5L, 1m10s

```
template<class P>
P lineProj(P a, P b, P p, bool refl=false) {
  P v = b - a;
  return p - v.perp()*(1+refl)*v.cross(p-a)/v.dist2();
} // 4b716b
```

Angle.h

Description: A class for ordering angles (as represented by int points and a number of rotations around the origin). Useful for rotational sweeping. Sometimes also represents points or vectors.

Usage: `vector<Angle> v = {w[0], w[0].t360() ...}; // sorted`
`int j = 0; rep(i,0,n) { while (v[j] < v[i].t180()) ++j; }`
sweeps j such that (j-i) represents the number of positively oriented triangles with vertices at 0 and i

6b951c, 35L, 11m20s

```
struct Angle {
  ll x, y;
  int t;
  Angle(ll x, ll y, int t=0) : x(x), y(y), t(t) {}
  Angle operator-(Angle b) const { return {x-b.x, y-b.y, t}; }
  int half() const {
    assert(x || y);
    return y < 0 || (y == 0 && x < 0);
  } // c935fb
  Angle t90() const { return {-y, x, t + (half() && x >= 0)}; }
  Angle t180() const { return {-x, -y, t + half()}; }
  Angle t360() const { return {x, y, t + 1}; }
}; // 88ad55
bool operator<(Angle a, Angle b) {
  // add a.dist2() and b.dist2() to also compare distances
  return make_tuple(a.t, a.half(), a.y * b.x) <
         make_tuple(b.t, b.half(), a.x * b.y);
} // 9c58ca

// Given two points, this calculates the smallest angle between
// them, i.e., the angle that covers the defined line segment.
pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
  if (b < a) swap(a, b);
  return (b < a.t180() ?
          make_pair(a, b) : make_pair(b, a.t360()));
} // 5eac29
Angle operator+(Angle a, Angle b) { // point a + vector b
  Angle r(a.x + b.x, a.y + b.y, a.t);
  if (a.t180() < r) r.t--;
  return r.t180() < a ? r.t360() : r;
} // 3d8073
Angle angleDiff(Angle a, Angle b) { // angle b - angle a
  int tu = b.t - a.t; a.t = b.t;
  return {a.x*b.x + a.y*b.y, a.x*b.y - a.y*b.x, tu - (b < a)};
} // ba3082
```

4.2 Circles

CircleIntersection.h

Description: Computes the pair of points at which two circles intersect. Returns false in case of no intersection.

"Point.h" 84d6d3, 11L, 3m50s

```
typedef Point<double> P;
bool circleInter(P a,P b,double r1,double r2,pair<P, P>* out) {
  if (a == b) { assert(r1 != r2); return false; }
  P vec = b - a;
  double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
         p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*d2;
  if (sum*sum < d2 || dif*dif > d2) return false;
  P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) / d2);
  *out = {mid + per, mid - per};
  return true;
} // c64785
```

CircleTangents.h

Description: Finds the external tangents of two circles, or internal if r2 is negated. Can return 0, 1, or 2 tangents – 0 if one circle contains the other (or overlaps it, in the internal case, or if the circles are the same); 1 if the circles are tangent to each other (in which case .first = .second and the tangent line is perpendicular to the line between the centers). .first and .second give the tangency points at circle 1 and 2 respectively. To find the tangents of a circle with a point set r2 to 0.

"Point.h" b0153d, 13L, 3m30s

```
template<class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2) {
  P d = c2 - c1;
  double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
  if (d2 == 0 || h2 < 0)  return {};
  vector<pair<P, P>> out;
  for (double sign : {-1, 1}) {
    P v = (d * dr + d.perp() * sqrt(h2) * sign) / d2;
    out.push_back({c1 + v * r1, c2 + v * r2});
  } // e25263
  if (h2 == 0) out.pop_back();
  return out;
} // 4835b9
```

CircleLine.h

Description: Finds the intersection between a circle and a line. Returns a vector of either 0, 1, or 2 intersection points. P is intended to be Point<double>.

"Point.h" e0cfba, 9L, 2m30s

```
template<class P>
vector<P> circleLine(P c, double r, P a, P b) {
  P ab = b - a, p = a + ab * (c-a).dot(ab) / ab.dist2();
  double s = a.cross(b, c), h2 = r*r - s*s / ab.dist2();
  if (h2 < 0) return {};
  if (h2 == 0) return {p};
  P h = ab.unit() * sqrt(h2);
  return {p - h, p + h};
} // 59aded
```

CirclePolygonIntersection.h

Description: Returns the area of the intersection of a circle with a ccw polygon.

Time: $\mathcal{O}(n)$

"../../content/geometry/Point.h" 19add1, 19L, 5m40s

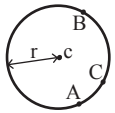
```
typedef Point<double> P;
#define arg(p, q) atan2(p.cross(q), p.dot(q))
double circlePoly(P c, double r, vector<P> ps) {
  auto tri = [&](P p, P q) {
    auto r2 = r * r / 2;
    P d = q - p;
    auto a = d.dot(p)/d.dist2(), b = (p.dist2()-r*r)/d.dist2();
    auto det = a * a - b;
    if (det <= 0) return arg(p, q) * r2;
    auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det));
    if (t < 0 || 1 <= s) return arg(p, q) * r2;
```

```
    P u = p + d * s, v = q + d * (t-1);
    return arg(p,u) * r2 + u.cross(v)/2 + arg(v,q) * r2;
  }; // a526fe
  auto sum = 0.0;
  rep(i,0,sz(ps))
    sum += tri(ps[i] - c, ps[(i + 1) % sz(ps)] - c);
  return sum;
} // f082e0
```

circumcircle.h

Description:

The circumcirle of a triangle is the circle intersecting all three vertices. ccRadius returns the radius of the circle going through points A, B and C and ccCenter returns the center of the same circle.



"Point.h" 1caa3a, 9L, 2m30s

```
typedef Point<double> P;
double ccRadius(const P& A, const P& B, const P& C) {
  return (B-A).dist()*(C-B).dist()*(A-C).dist()/
      abs((B-A).cross(C-A))/2;
} // 607d98
P ccCenter(const P& A, const P& B, const P& C) {
  P b = C-A, c = B-A;
  return A + (b*c.dist2()-c*b.dist2()).perp()/b.cross(c)/2;
} // 79372e
```

MinimumEnclosingCircle.h

Description: Computes the minimum circle that encloses a set of points.
Time: expected $\mathcal{O}(n)$

"circumcircle.h" 09dd0a, 17L, 4m

```
pair<P, double> mec(vector<P> ps) {
  shuffle(all(ps), mt19937(time(0)));
  P o = ps[0];
  double r = 0, EPS = 1 + 1e-8;
  rep(i,0,sz(ps)) if ((o - ps[i]).dist() > r * EPS) {
    o = ps[i], r = 0;
    rep(j,0,i) if ((o - ps[j]).dist() > r * EPS) {
      o = (ps[i] + ps[j]) / 2;
      r = (o - ps[i]).dist();
      rep(k,0,j) if ((o - ps[k]).dist() > r * EPS) {
        o = ccCenter(ps[i], ps[j], ps[k]);
        r = (o - ps[i]).dist();
      } // 64802f
    } // 7b0ecf
  } // dcfd0e
  return {o, r};
} // 09dd0a
```

4.3 Polygons

InsidePolygon.h

Description: Returns true if p lies within the polygon. If strict is true, it returns false for points on the boundary. The algorithm uses products in intermediate steps so watch out for overflow.
Usage: `vector<P> v = {P{4,4}, P{1,2}, P{2,1}};`
`bool in = inPolygon(v, P{3, 3}, false);`
Time: $\mathcal{O}(n)$

"Point.h", "OnSegment.h", "SegmentDistance.h" 2bf504, 11L, 2m50s

```
template<class P>
bool inPolygon(vector<P> &p, P a, bool strict = true) {
  int cnt = 0, n = sz(p);
  rep(i,0,n) {
    P q = p[(i + 1) % n];
    if (onSegment(p[i], q, a)) return !strict;
    //or: if (segDist(p[i], q, a) <= eps) return !strict;
    cnt ^= ((a.y<p[i].y) - (a.y<q.y)) * a.cross(p[i], q) > 0;
  } // 1b9961
  return cnt;
} // c7225e
```

PolygonCenter.h

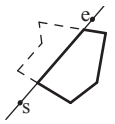
Description: Returns the center of mass for a polygon.
Time: $\mathcal{O}(n)$

"Point.h" 9706dc, 9L, 2m10s

```
typedef Point<double> P;
P polygonCenter(const vector<P>& v) {
  P res(0, 0); double A = 0;
  for (int i = 0, j = sz(v) - 1; i < sz(v); j = i++) {
    res = res + (v[i] + v[j]) * v[j].cross(v[i]);
    A += v[j].cross(v[i]);
  } // 307102
  return res / A / 3;
} // 0d0d84
```

PolygonCut.h

Description:
Returns a vector with the vertices of a polygon with everything to the left of the line going from s to e cut away.
Usage: `vector<P> p = ...;`
`p = polygonCut(p, P(0,0), P(1,0));`



"Point.h" d07181, 13L, 3m10s

```
typedef Point<double> P;
vector<P> polygonCut(const vector<P>& poly, P s, P e) {
  vector<P> res;
  rep(i,0,sz(poly)) {
    P cur = poly[i], prev = i ? poly[i-1] : poly.back();
    auto a = s.cross(e, cur), b = s.cross(e, prev);
    if ((a < 0) != (b < 0))
      res.push_back(cur + (prev - cur) * (a / (a - b)));
    if (a < 0)
      res.push_back(cur);
  } // 757c0d
  return res;
} // 42c993
```

PolygonUnion.h

Description: Calculates the area of the union of n polygons (not necessarily convex). The points within each polygon must be given in CCW order. (Epsilon checks may optionally be added to sideOf/sgn, but shouldn't be needed.)
Time: $\mathcal{O}\left(N^2\right)$, where N is the total number of points

"Point.h", "sideOf.h" 3931c6, 33L, 10m20s

```
typedef Point<double> P;
double rat(P a, P b) { return sgn(b.x) ? a.x/b.x : a.y/b.y; }
double polyUnion(vector<vector<P>>& poly) {
  double ret = 0;
  rep(i,0,sz(poly)) rep(v,0,sz(poly[i])) {
    P A = poly[i][v], B = poly[i][(v + 1) % sz(poly[i])];
    vector<pair<double, int>> segs = {{0, 0}, {1, 0}};
    rep(j,0,sz(poly)) if (i != j) {
      rep(u,0,sz(poly[j])) {
        P C = poly[j][u], D = poly[j][(u + 1) % sz(poly[j])];
        int sc = sideOf(A, B, C), sd = sideOf(A, B, D);
        if (sc != sd) {
          double sa = C.cross(D, A), sb = C.cross(D, B);
          if (min(sc, sd) < 0)
            segs.emplace_back(sa / (sa - sb), sgn(sc - sd));
        } else if (!sc && !sd && j<i && sgn((B-A).dot(D-C))>0){
            // cf73dd
          segs.emplace_back(rat(C - A, B - A), 1);
          segs.emplace_back(rat(D - A, B - A), -1);
        } // 313ead
      } // 0d1e2c
    } // fdc855
    sort(all(segs));
    for (auto& s : segs) s.first = min(max(s.first, 0.0), 1.0);
    double sum = 0;
    int cnt = segs[0].second;
    rep(j,1,sz(segs)) {
      if (!cnt) sum += segs[j].first - segs[j - 1].first;
      cnt += segs[j].second;
    } // f5884f
    ret += A.cross(B) * sum;
  } // 19180c
  return ret / 2;
} // 6e88cf
```

ConvexHull.h

Description:
Returns a vector of the points of the convex hull in counter-clockwise order. Points on the edge of the hull between two other points are not considered part of the hull. To include these "borders", follow the code comments.
Time: $\mathcal{O}(n \log n)$

"Point.h" 310954, 13L, 3m50s

```
typedef Point<ll> P;
vector<P> convexHull(vector<P> pts) {
  if (sz(pts) <= 1) return pts;
  sort(all(pts));
  vector<P> h(sz(pts)+1); // 2*sz(pts) to include borders
  int s = 0, t = 0;
  for (int it = 2; it--; s = --t, reverse(all(pts)))
    for (P p : pts) {
      while (t >= s + 2 && h[t-2].cross(h[t-1], p) <= 0) t--;
      h[t++] = p; // < above to include borders
    } // bf0344
  return {h.begin(), h.begin() + t - (t == 2 && h[0] == h[1])};
} // ec85f8
```

HullDiameter.h

Description: Returns the two points with max distance on a convex hull (ccw, no duplicate/collinear points).
Time: $\mathcal{O}(n)$

"Point.h" c571b8, 12L, 3m

```
typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
  int n = sz(S), j = n < 2 ? 0 : 1;
  pair<ll, array<P, 2>> res({0, {S[0], S[0]}});
  rep(i,0,j)
    for (;; j = (j + 1) % n) {
      res = max(res, {(S[i] - S[j]).dist2(), {S[i], S[j]}});
      if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0)
        break;
    } // 56cc40
  return res.second;
} // 5f726b
```

PointInsideHull.h

Description: Determine whether a point t lies inside a convex hull (CCW order, with no collinear points). Returns true if point lies within the hull. If strict is true, points on the boundary aren't included.
Time: $\mathcal{O}(\log N)$

"Point.h", "sideOf.h", "OnSegment.h" 71446b, 14L, 4m

```
typedef Point<ll> P;

bool inHull(const vector<P>& l, P p, bool strict = true) {
  int a = 1, b = sz(l) - 1, r = !strict;
  if (sz(l) < 3) return r && onSegment(l[0], l.back(), p);
```

```
  if (sideOf(l[0], l[a], l[b]) > 0) swap(a, b);
  if (sideOf(l[0], l[a], p) >= r || sideOf(l[0], l[b], p)<= -r)
    return false;
  while (abs(a - b) > 1) {
    int c = (a + b) / 2;
    (sideOf(l[0], l[c], p) > 0 ? b : a) = c;
  } // b265ab
  return sgn(l[a].cross(l[b], p)) < r;
} // c74639
```

LineHullIntersection.h

Description: Line-convex polygon intersection. The polygon must be ccw and have no collinear points. lineHull(line, poly) returns a pair describing the intersection of a line with the polygon: $\bullet\ (-1,-1)$ if no collision, $\bullet\ (i,-1)$ if touching the corner i, $\bullet\ (i,i)$ if along side $(i,i+1)$, $\bullet\ (i,j)$ if crossing sides $(i,i+1)$ and $(j,j+1)$. In the last case, if a corner i is crossed, this is treated as happening on side $(i,i+1)$. The points are returned in the same order as the line hits the polygon. `extrVertex` returns the point of a hull with the max projection onto a line.
Time: $\mathcal{O}(\log n)$

"Point.h" 7cf45b, 38L, 11m30s

```
#define cmp(i,j) sgn(dir.perp().cross(poly[(i)%n]-poly[(j)%n]))
#define extr(i) cmp(i + 1, i) >= 0 && cmp(i, i - 1 + n) < 0
template <class P> int extrVertex(vector<P>& poly, P dir) {
  int n = sz(poly), lo = 0, hi = n;
  if (extr(0)) return 0;
  while (lo + 1 < hi) {
    int m = (lo + hi) / 2;
    if (extr(m)) return m;
    int ls = cmp(lo + 1, lo), ms = cmp(m + 1, m);
    (ls < ms || (ls == ms && ls == cmp(lo, m)) ? hi : lo) = m;
  } // 68a24c
  return lo;
} // 7f0477
#define cmpL(i) sgn(a.cross(poly[i], b))
template <class P>
array<int, 2> lineHull(P a, P b, vector<P>& poly) {
  int endA = extrVertex(poly, (a - b).perp());
  int endB = extrVertex(poly, (b - a).perp());
  if (cmpL(endA) < 0 || cmpL(endB) > 0)
    return {-1, -1};
  array<int, 2> res;
  rep(i,0,2) {
    int lo = endB, hi = endA, n = sz(poly);
    while ((lo + 1) % n != hi) {
      int m = ((lo + hi + (lo < hi ? 0 : n)) / 2) % n;
      (cmpL(m) == cmpL(endB) ? lo : hi) = m;
    } // 52528c
    res[i] = (lo + !cmpL(hi)) % n;
    swap(endA, endB);
  } // c05c70
  if (res[0] == res[1]) return {res[0], -1};
  if (!cmpL(res[0]) && !cmpL(res[1]))
    switch ((res[0] - res[1] + sz(poly) + 1) % sz(poly)) {
      case 0: return {res[0], res[0]};
      case 2: return {res[1], res[1]};
    } // 8fa383
  return res;
} // 36fc8e
```

PolygonContainmentTree.h

Description: building tree of polygon containment
Memory: $\mathcal{O}(N)$
Time: $\mathcal{O}(N \log N)$

14c82a, 44L, 12m40s

```
struct P { ll x, y; };

int current_x;

struct Segment {
  int idx; P p1, p2; bool is_upper;
  Segment(P p, P q, int i): idx(i), p1(p), p2(q), is_upper(p2.x
       < p1.x) { if (is_upper)swap(p1, p2); }
  ld get_y(ll x) const { return (ld) (p2.y - p1.y) / (p2.x - p1
       .x) * (x - p1.x) + p1.y; }
  tuple<ld, bool, int> get_comp() const { return {get_y(
       current_x), is_upper, p2.x}; }
  bool operator<(const Segment & o) const { return get_comp() <
        o.get_comp(); }
}; // fc8b4f

vector<int> build(vector<vector<P>>& polygons) {
  int n = sz(polygons);
  vector<tuple<int, int, int, Segment>> edges; // polygon edges
  rep(idx, 0, n) {
    const auto & v = polygons[idx];
    rep(i, 0, sz(v)) {
      int j = (i + 1) % sz(v);
      if (v[i].x == v[j].x)continue; // ignores vertical edges
      Segment seg = Segment(v[i], v[j], idx);
      edges.eb(seg.p1.x, 0, -seg.p1.y, seg);
      edges.eb(seg.p2.x, 1, -seg.p2.y, seg);
    } // b28b76
  } // 2603da
  sort(edges.begin(), edges.end());
  set<Segment> s;
  vector pai(n+1, n), vis(n, 0);
  for (auto [l, t, y, seg]: edges) {
    current_x = l;
    int i = seg.idx;
    if (t == 0) {
      if (not vis[i]) {
        vis[i] = true;
        auto it = s.upper_bound(seg);
        if (it == s.end())pai[i] = n;
        else if (it->is_upper)pai[i] = it->idx;
        else pai[i] = pai[it->idx];
      } // 17d107
      s.insert(seg);
    } // aff5b0
    else s.erase(seg);
  } // 3514ff
  return pai;
} // 0b7cbd
```

Minkowski.h

Description: Calculates the Minkowski sum of two convex Polygons in ccw order
Time: $\mathcal{O}(N+M)$

"Point.h" e510cd, 19L, 5m50s

```
template<typename P>
vector<P> minkowski(vector<P> p, vector<P> q){
  auto comp = [](auto a, auto b) {
      return make_pair(a.y, a.x) < make_pair(b.y, b.x);
  }; // 278e98
    rotate(p.begin(), min_element(all(p), comp), p.end());
    rotate(q.begin(), min_element(all(q), comp), q.end());
    p.pb(p[0]); p.pb(p[1]);
    q.pb(q[0]); q.pb(q[1]);
    vector<P> result;
    int i = 0, j = 0;
    while(i < sz(p) - 2 || j < sz(q) - 2){
        result.pb(p[i] + q[j]);
        auto cross = (p[i + 1] - p[i]).cross(q[j + 1] - q[j]);
        if(cross >= 0 && i < sz(p) - 2) i++;
        if(cross <= 0 && j < sz(q) - 2) j++;
    } // 57f2d7
    return result;
} // a99371
```

HalfPlane.h

Description: Calculates the halfplane intersection of Halfplanes, Halfplanes allow everything to the left. Be careful with precision. Intersects with an artificial $[-INF, INF]^2$ bounding box.
Time: $\mathcal{O}(N \log N)$

"Point.h" 0a6375, 50L, 15m50s

```
const ld EPS = 1e-9, INF = 1e9;
typedef Point<ld> P;
struct HP {
    P p, pq;
  ld angle;
    HP() {}
    HP(P a, P b) : p(a), pq(b - a), angle(atan2(pq.y, pq.x)) {}

    bool out(const P & r) { return pq.cross(r - p) < -EPS; }
    bool operator<(const HP & e) const { return angle < e.angle
        ; }
    friend P inter(const HP & s, const HP & t) {
        ld alpha = (t.p - s.p).cross(t.pq) / s.pq.cross(t.pq);
        return s.p + (s.pq * alpha);
    } // 946b1a
}; // 782415
vector<P> hp_intersect(vector<HP> H) {
    P box[4] = {
        P(INF, INF),
        P(-INF, INF),
        P(-INF, -INF),
        P(INF, -INF)
    }; // 63b3ab
    for(int i = 0; i < 4; i++) H.eb(box[i], box[(i+1) % 4]);
    sort(H.begin(), H.end());
    deque<HP> dq;
    int len = 0;
    rep(i, 0, sz(H)) {
        while (len > 1 && H[i].out(inter(dq[len-1], dq[len-2]))
            )
            dq.pop_back(), len--;
        while (len > 1 && H[i].out(inter(dq[0], dq[1])))
            dq.pop_front(), len--;
    // Special case check: Parallel half planes
        if (len > 0 && fabsl(H[i].pq.cross(dq[len-1].pq)) < EPS
            ) {
            if (H[i].pq.dot(dq[len-1].pq) < 0.0) return {};
            if (H[i].out(dq[len-1].p))
                dq.pop_back(), len--;
            else continue;
        } // 6ef632
        dq.push_back(H[i]); len++;
    } // 82e81e
    while (len > 2 && dq[0].out(inter(dq[len-1], dq[len-2])))
        dq.pop_back(), len--;
    while (len > 2 && dq[len-1].out(inter(dq[0], dq[1])))
    dq.pop_front(), len--;
    if (len < 3) return {};
    vector<P> ret(len);
  rep(i, 0, len-1) ret[i] = inter(dq[i], dq[i+1]);
    ret.back() = inter(dq[len-1], dq[0]);
    return ret;
} // 8ab567
```

4.4 Misc. Point Set Problems

ClosestPair.h

Description: Finds the closest pair of points.
Time: $\mathcal{O}(n \log n)$

"Point.h" ac41a6, 17L, 4m20s

```
typedef Point<ll> P;
pair<P, P> closest(vector<P> v) {
  assert(sz(v) > 1);
  set<P> S;
  sort(all(v), [](P a, P b) { return a.y < b.y; });
  pair<ll, pair<P, P>> ret{LLONG_MAX, {P(), P()}};
  int j = 0;
  for (P p : v) {
    P d{1 + (ll)sqrt(ret.first), 0};
    while (v[j].y <= p.y - d.x) S.erase(v[j++]);
    auto lo = S.lower_bound(p - d), hi = S.upper_bound(p + d);
    for (; lo != hi; ++lo)
      ret = min(ret, {(*lo - p).dist2(), {*lo, p}});
    S.insert(p);
  } // 5b096c
  return ret.second;
} // bf22c6
```

ManhattanMST.h

Description: Given N points, returns up to 4*N edges, which are guaranteed to contain a minimum spanning tree for the graph with edge weights w(p, q) = —p.x - q.x— + —p.y - q.y—. Edges are in the form (distance, src, dst). Use a standard MST algorithm on the result to find the final MST.
Time: $\mathcal{O}(N \log N)$

"Point.h" ee6376, 23L, 5m40s

```
typedef Point<ll> P;
vector<array<ll, 3>> manhattanMST(vector<P> ps) {
  vi id(sz(ps));
  iota(all(id), 0);
  vector<array<ll, 3>> edges;
  rep(k,0,4) {
    sort(all(id), [&](int i, int j) {
         return (ps[i]-ps[j]).x < (ps[j]-ps[i]).y;}); // 02bd87
    map<ll, int> sweep;
    for (int i : id) {
      for (auto it = sweep.lower_bound(-ps[i].y);
                it != sweep.end(); sweep.erase(it++)) {
        int j = it->second;
        P d = ps[i] - ps[j];
        if (d.y > d.x) break;
        edges.push_back({d.y + d.x, i, j});
      } // 271a42
      sweep[-ps[i].y] = i;
    } // e692f6
    for (P& p : ps) if (k & 1) p.x = -p.x; else swap(p.x, p.y);
  } // efbf43
  return edges;
} // babb49
```

FastDelaunay.h

Description: Fast Delaunay triangulation. Each circumcircle contains none of the input points. There must be no duplicate points. If all points are on a line, no triangles will be returned. Should work for doubles as well, though there may be precision issues in 'circ'. Returns triangles in order {t[0][0], t[0][1], t[0][2], t[1][0], ... }, all counter-clockwise.
Time: $\mathcal{O}(n \log n)$

"Point.h" eefdf5, 82L, 26m10s

```
typedef Point<ll> P;
typedef struct Quad* Q;
typedef __int128_t lll; // (can be ll if coords are < 2e4)
P arb(LLONG_MAX,LLONG_MAX); // not equal to any other point
struct Quad {
  Q rot, o; P p = arb; bool mark;
  P& F() { return r()->p; }
  Q& r() { return rot->rot; }
  Q prev() { return rot->o->rot; }
  Q next() { return r()->prev(); }
} *H; // 18059e
bool circ(P p, P a, P b, P c) { // is p in the circumcircle?
  lll p2 = p.dist2(), A = a.dist2()-p2,
      B = b.dist2()-p2, C = c.dist2()-p2;
  return p.cross(a,b)*C + p.cross(b,c)*A + p.cross(c,a)*B > 0;
} // 6aff7b
Q makeEdge(P orig, P dest) {
  Q r = H ? H : new Quad{new Quad{new Quad{new Quad{0}}}};
  H = r->o; r->r()->r() = r;
  rep(i,0,4) r = r->rot, r->p = arb, r->o = i & 1 ? r : r->r();
  r->p = orig; r->F() = dest;
  return r;
} // b3b5b1
void splice(Q a, Q b) {
  swap(a->o->rot->o, b->o->rot->o); swap(a->o, b->o);
} // 86ce01
Q connect(Q a, Q b) {
  Q q = makeEdge(a->F(), b->p);
  splice(q, a->next());
  splice(q->r(), b);
  return q;
} // 4a4fc2
pair<Q,Q> rec(const vector<P>& s) {
  if (sz(s) <= 3) {
    Q a = makeEdge(s[0], s[1]), b = makeEdge(s[1], s.back());
    if (sz(s) == 2) return { a, a->r() };
    splice(a->r(), b);
    auto side = s[0].cross(s[1], s[2]);
    Q c = side ? connect(b, a) : 0;
    return {side < 0 ? c->r() : a, side < 0 ? c : b->r() };
  } // c9e598
#define H(e) e->F(), e->p
#define valid(e) (e->F().cross(H(base)) > 0)
  Q A, B, ra, rb;
  int half = sz(s) / 2;
  tie(ra, A) = rec({all(s) - half});
  tie(B, rb) = rec({sz(s) - half + all(s)});
  while ((B->p.cross(H(A)) < 0 && (A = A->next())) ||
         (A->p.cross(H(B)) > 0 && (B = B->r()->o)));
  Q base = connect(B->r(), A);
  if (A->p == ra->p) ra = base->r();
  if (B->p == rb->p) rb = base;
#define DEL(e, init, dir) Q e = init->dir; if (valid(e)) \
    while (circ(e->dir->F(), H(base), e->F())) { \
      Q t = e->dir; \
      splice(e, e->prev()); \
      splice(e->r(), e->r()->prev()); \
      e->o = H; H = e; e = t; \
    } // a2e9b5
  for (;;) {
    DEL(LC, base->r(), o);  DEL(RC, base, prev());
    if (!valid(LC) && !valid(RC)) break;
    if (!valid(LC) || (valid(RC) && circ(H(RC), H(LC))))
      base = connect(RC, base->r());
    else
      base = connect(base->r(), LC->r());
  } // fcf7ef
  return { ra, rb };
} // 7cf639
vector<P> triangulate(vector<P> pts) {
  sort(all(pts));  assert(unique(all(pts)) == pts.end());
  if (sz(pts) < 2) return {};
  Q e = rec(pts).first;
  vector<Q> q = {e};
  int qi = 0;
  while (e->o->F().cross(e->F(), e->p) < 0) e = e->o;
#define ADD { Q c = e; do { c->mark = 1; pts.push_back(c->p); \
  q.push_back(c->r()); c = c->next(); } while (c != e); } // 43e195 // 43e195
  ADD; pts.clear();
  while (qi < sz(q)) if (!(e = q[qi++])->mark) ADD;
  return pts;
} // a02307
```

4.5 3D

PolyhedronVolume.h

Description: Magic formula for the volume of a polyhedron. Faces should point outwards.

3058c3, 6L, 1m30s

```
template<class V, class L>
double signedPolyVolume(const V& p, const L& trilist) {
  double v = 0;
  for (auto i : trilist) v += p[i.a].cross(p[i.b]).dot(p[i.c]);
  return v / 6;
} // fca9df
```

Point3D.h

Description: Class to handle points in 3D space. T can be e.g. double or long long.

8058ae, 32L, 13m10s

```
template<class T> struct Point3D {
  typedef Point3D P;
  typedef const P& R;
  T x, y, z;
  explicit Point3D(T x=0, T y=0, T z=0) : x(x), y(y), z(z) {}
  bool operator<(R p) const {
    return tie(x, y, z) < tie(p.x, p.y, p.z); } // 8eef6b
  bool operator==(R p) const {
    return tie(x, y, z) == tie(p.x, p.y, p.z); } // bd6a08
  P operator+(R p) const { return P(x+p.x, y+p.y, z+p.z); }
  P operator-(R p) const { return P(x-p.x, y-p.y, z-p.z); }
  P operator*(T d) const { return P(x*d, y*d, z*d); }
  P operator/(T d) const { return P(x/d, y/d, z/d); }
  T dot(R p) const { return x*p.x + y*p.y + z*p.z; }
  P cross(R p) const {
    return P(y*p.z - z*p.y, z*p.x - x*p.z, x*p.y - y*p.x);
  } // a77b7e
  T dist2() const { return x*x + y*y + z*z; }
  double dist() const { return sqrt((double)dist2()); }
  //Azimuthal angle (longitude) to x-axis in interval [-pi, pi]
  double phi() const { return atan2(y, x); }
  //Zenith angle (latitude) to the z-axis in interval [0, pi]
  double theta() const { return atan2(sqrt(x*x+y*y),z); }
  P unit() const { return *this/(T)dist(); } //makes dist()=1
  //returns unit vector normal to *this and p
  P normal(P p) const { return cross(p).unit(); }
  //returns point rotated 'angle' radians ccw around axis
  P rotate(double angle, P axis) const {
    double s = sin(angle), c = cos(angle); P u = axis.unit();
    return u*dot(u)*(1-c) + (*this)*c - cross(u)*s;
  } // 73af70
}; // 8058ae
```

3dHull.h

Description: Computes all faces of the 3-dimension hull of a point set. *No four points must be coplanar*, or else random results will be returned. All faces will point outwards.
Time: $\mathcal{O}(n^2)$

"Point3D.h" 5b45fc, 46L, 11m20s

```
typedef Point3D<double> P3;
struct PR {
  void ins(int x) { (a == -1 ? a : b) = x; }
```

```
  void rem(int x) { (a == x ? a : b) = -1; }
  int cnt() { return (a != -1) + (b != -1); }
  int a, b;
}; // cf7c9e
struct F { P3 q; int a, b, c; };
vector<F> hull3d(const vector<P3>& A) {
  assert(sz(A) >= 4);
  vector<vector<PR>> E(sz(A), vector<PR>(sz(A), {-1, -1}));
#define E(x,y) E[f.x][f.y]
  vector<F> FS;
  auto mf = [&](int i, int j, int k, int l) {
    P3 q = (A[j] - A[i]).cross((A[k] - A[i]));
    if (q.dot(A[l]) > q.dot(A[i]))
      q = q * -1;
    F f{q, i, j, k};
    E(a,b).ins(k); E(a,c).ins(j); E(b,c).ins(i);
    FS.push_back(f);
  }; // d73a06
  rep(i,0,4) rep(j,i+1,4) rep(k,j+1,4)
    mf(i, j, k, 6 - i - j - k);

  rep(i,4,sz(A)) {
    rep(j,0,sz(FS)) {
      F f = FS[j];
      if(f.q.dot(A[i]) > f.q.dot(A[f.a])) {
        E(a,b).rem(f.c);
        E(a,c).rem(f.b);
        E(b,c).rem(f.a);
        swap(FS[j--], FS.back());
        FS.pop_back();
      } // 5cd5dc
    } // 220067
    int nw = sz(FS);
    rep(j,0,nw) {
      F f = FS[j];
#define C(a, b, c) if (E(a,b).cnt() != 2) mf(f.a, f.b, i, f.c);
      C(a, b, c); C(a, c, b); C(b, c, a);
    } // 248ed4
  } // 47289c
  for (F& it : FS) if ((A[it.b] - A[it.a]).cross(
    A[it.c] - A[it.a]).dot(it.q) <= 0) swap(it.c, it.b);
  return FS;
}; // be2ca2
```

sphericalDistance.h

Description: Returns the shortest distance on the sphere with radius radius between the points with azimuthal angles (longitude) f1 (ϕ_1) and f2 (ϕ_2) from x axis and zenith angles (colatitude) t1 (θ_1) and t2 (θ_2) from z axis (0 = north pole). All angles measured in radians. The algorithm starts by converting the spherical coordinates to cartesian coordinates so if that is what you have you can use only the two last rows. dx*radius is then the difference between the two points in the x direction and d*radius is the total distance between the points.

80b54c, 8L, 2m40s

```
double sphericalDistance(double f1, double t1,
    double f2, double t2, double radius) {
  double dx = sin(t2)*cos(f2) - sin(t1)*cos(f1);
  double dy = sin(t2)*sin(f2) - sin(t1)*sin(f1);
  double dz = cos(t2) - cos(t1);
  double d = sqrt(dx*dx + dy*dy + dz*dz);
  return radius*2*asin(min(1.,d/2));
} // cb1a4b
```

Graph (5)

5.1 Miscellaneous

BellmanFord.h

Description: Calculates shortest paths from s in a graph that might have negative edge weights. Unreachable nodes get dist = inf; nodes reachable through negative-weight cycles get dist = -inf. Assumes $V^2 \max|w_i| < \sim 2^{63}$. Leaves eds sorted.
Time: $\mathcal{O}(VE)$

830a8f, 21L, 6m10s

```
const ll inf = LLONG_MAX;
struct Ed { int a, b, w, s() { return a < b ? a : -a; }};
struct Node { ll dist = inf; int prev = -1; };
void bellmanFord(vector<Node>& nodes, vector<Ed>& eds, int s) {
  nodes[s].dist = 0;
  sort(all(eds), [](Ed a, Ed b) { return a.s() < b.s(); });
  int lim = sz(nodes) / 2 + 2; // /3+100 with shuffled vertices
  rep(i,0,lim) for (Ed ed : eds) {
    Node cur = nodes[ed.a], &dest = nodes[ed.b];
    if (abs(cur.dist) == inf) continue;
    ll d = cur.dist + ed.w;
    if (d < dest.dist) {
      dest.prev = ed.a;
      dest.dist = (i < lim-1 ? d : -inf);
    } // 452019
  } // 75a370
  rep(i,0,lim) for (Ed e : eds) {
    if (nodes[e.a].dist == -inf)
      nodes[e.b].dist = -inf;
  } // 1d7315
} // fa39de
```

FunctGraph.h

Description: Functional graph. head[v] is the cycle vertex reached by v, comp[v] is its component/cycle index, cycles contains every directed cycle.
Memory: $\mathcal{O}(N)$
Time: $\mathcal{O}(N)$

2b2048, 24L, 5m10s

```
struct FunctGraph {
  int n; vi head, comp; vector<vi> cycles;
  FunctGraph(const vi& f):n(sz(f)),head(n,-1),comp(n,-1){
    vi deg(n), q;
    for(int v : f)deg[v]++;
    q.reserve(n);
    rep(v,0,n)if (!deg[v])q.pb(v);
    rep(i,0,sz(q)){
      int v=q[i];
      if(!--deg[f[v]])q.pb(f[v]);
    } // 282d74
    rep(v,0,n)if (deg[v] && head[v] < 0){
      int c=sz(cycles); vi cyc;
      for(int u = v; head[u] < 0; u = f[u])
        head[u] = u, comp[u] = c, cyc.pb(u);
      cycles.pb(move(cyc));
    } // 65f9c9
    for(int i = sz(q); i--;){
      int v=q[i];
      head[v]=head[f[v]];
      comp[v]=comp[f[v]];
    } // 2750a0
  } // ce6083
}; // 2b2048
```

DirectedMST.h

Description: Finds a minimum spanning tree/arborescence of a directed graph, given a root node. If no MST exists, returns -1.
Time: $\mathcal{O}(E \log V)$

"../data-structures/UnionFindRollback.h" 39e620, 58L, 15m10s

```
struct Edge { int a, b; ll w; };
struct Node {
  Edge key;
  Node *l, *r;
  ll delta;
  void prop() {
    key.w += delta;
    if (l) l->delta += delta;
    if (r) r->delta += delta;
    delta = 0;
  } // 0d348f
  Edge top() { prop(); return key; }
}; // ab4902
Node *merge(Node *a, Node *b) {
  if (!a || !b) return a ?: b;
  a->prop(), b->prop();
  if (a->key.w > b->key.w) swap(a, b);
  swap(a->l, (a->r = merge(b, a->r)));
  return a;
} // c5109e
void pop(Node*& a) { a->prop(); a = merge(a->l, a->r); }
pair<ll, vi> dmst(int n, int r, vector<Edge>& g) {
  RollbackUF uf(n);
  vector<Node*> heap(n);
  for (Edge e : g) heap[e.b] = merge(heap[e.b], new Node{e});
  ll res = 0;
  vi seen(n, -1), path(n), par(n);
  seen[r] = r;
  vector<Edge> Q(n), in(n, {-1,-1}), comp;
  deque<tuple<int, int, vector<Edge>>> cycs;
  rep(s,0,n) {
    int u = s, qi = 0, w;
    while (seen[u] < 0) {
      if (!heap[u]) return {-1,{}};
      Edge e = heap[u]->top();
      heap[u]->delta -= e.w, pop(heap[u]);
      Q[qi] = e, path[qi++] = u, seen[u] = s;
      res += e.w, u = uf.find(e.a);
      if (seen[u] == s) {
        Node* cyc = 0;
        int end = qi, time = uf.time();
        do cyc = merge(cyc, heap[w = path[--qi]]);
        while (uf.join(u, w));
        u = uf.find(u), heap[u] = cyc, seen[u] = -1;
        cycs.push_front({u, time, {&Q[qi], &Q[end]}});
      } // 00a339
    } // c8f0da
    rep(i,0,qi) in[uf.find(Q[i].b)] = Q[i];
  } // fa3c2c
  for (auto& [u,t,comp] : cycs) { // restore sol (optional)
    uf.rollback(t);
    Edge inEdge = in[u];
    for (auto& e : comp) in[uf.find(e.b)] = e;
    in[uf.find(inEdge.b)] = inEdge;
  } // 4f9b56
  rep(i,0,n) par[i] = in[i].a;
  return {res, par};
} // efa3a4
```

EulerWalk.h

Description: Eulerian undirected/directed path/cycle algorithm. Input should be a vector of (dest, global edge index), where for undirected graphs, forward/backward edges have the same index. Returns a list of nodes in the Eulerian path/cycle with src at both start and end, or empty list if no cycle/path exists. To get edge indices back, add .second to s and ret.
Time: $\mathcal{O}(V + E)$

780b64, 15L, 4m40s

```
vi eulerWalk(vector<vector<pii>>& gr, int nedges, int src=0) {
  int n = sz(gr);
  vi D(n), its(n), eu(nedges), ret, s = {src};
  D[src]++; // to allow Euler paths, not just cycles
  while (!s.empty()) {
```

```
    int x = s.back(), y, e, &it = its[x], end = sz(gr[x]);
    if (it == end){ ret.push_back(x); s.pop_back(); continue; }
    tie(y, e) = gr[x][it++];
    if (!eu[e]) {
      D[x]--, D[y]++;
      eu[e] = 1; s.push_back(y);
    }} // 22a87a // 94de26
  for (int x : D) if (x < 0 || sz(ret) != nedges+1) return {};
  return {ret.rbegin(), ret.rend()};
} // 780b64
```

5.2 Flow

Dinic.h

Description: Flow algorithm with complexity $O(VE\log U)$ where $U = \max|\text{cap}|$. Implements capacity scaling. $O(\min(E^{1/2}, V^{2/3})E)$ if $U = 1$; $O(\sqrt{V}E)$ for bipartite matching.

7665a7, 43L, 11m30s

```
struct Dinic {
  struct Edge {
    int to, rev;
    ll c, oc;
    ll flow() { return max(oc - c, 0LL); } // if you need flows
  }; // d46531
  vi lvl, ptr, q;
  vector<vector<Edge>> adj;
  Dinic(int n) : lvl(n), ptr(n), q(n), adj(n) {}
  void addEdge(int a, int b, ll c, ll rcap = 0) {
    adj[a].push_back({b, sz(adj[b]), c, c});
    adj[b].push_back({a, sz(adj[a]) - 1, rcap, rcap});
  } // 01ad91
  ll dfs(int v, int t, ll f) {
    if (v == t || !f) return f;
    for (int& i = ptr[v]; i < sz(adj[v]); i++) {
      Edge& e = adj[v][i];
      if (lvl[e.to] == lvl[v] + 1)
        if (ll p = dfs(e.to, t, min(f, e.c))) {
          e.c -= p, adj[e.to][e.rev].c += p;
          return p;
        } // 494522
    } // f73941
    return 0;
  } // 0922b8
  ll calc(int s, int t) {
    ll flow = 0; q[0] = s;
    rep(L,0,31) do { // 'int L=30' maybe faster for random data
      lvl = ptr = vi(sz(q));
      int qi = 0, qe = lvl[s] = 1;
      while (qi < qe && !lvl[t]) {
        int v = q[qi++];
        for (const Edge & e : adj[v])
          if (!lvl[e.to] && e.c >> (30 - L))
            q[qe++] = e.to, lvl[e.to] = lvl[v] + 1;
      } // 1f1250
      while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
    } while (lvl[t]); // 4a7fc1
    return flow;
  } // 801ca9
  bool leftOfMinCut(int a) { return lvl[a] != 0; }
  void reset(){for(auto& a:adj)for(auto& e:a)e.c=e.oc;}
}; // 7665a7
```

MinCostMaxFlow.h

Description: Min-cost max-flow with potentials. Reuses each Dijkstra to push flow through the shortest-path subgraph. If initial costs can be negative, call setpi(s) first. Negative cost cycles are unsupported. Positive edge.flow values give the resulting flow.

Time: $\mathcal{O}(FElogV)$ worst case. $\mathcal{O}(VE)$ for setpi.

<bits/extc++.h> 20ade0, 59L, 17m

```
struct MCMF {
  static constexpr ll INF=LLONG_MAX/4;
  struct edge { int to,rev; ll cap,cost,flow; };
  using PQ=__gnu_pbds::priority_queue<pair<ll,int>>;
  int n; vector<vector<edge>> ed; vector<ll> dist,pi;
  vi ptr; vector<char> on; vector<PQ::point_iterator> its;
  MCMF(int _n):n(_n),ed(n),dist(n),pi(n),ptr(n),on(n),its(n){}
  void addEdge(int a,int b,ll cap,ll cost){
    ed[a].pb({b,sz(ed[b]),cap,cost,0});
    ed[b].pb({a,sz(ed[a])-1,0,-cost,0});
  } // 43f268
  ll rem(const edge& e){return e.cap-e.flow;}
  void setpi(int s){
    fill(all(pi),INF); pi[s]=0;
    rep(k,0,n){
      bool ch=0;
      rep(v,0,n)if(pi[v]<INF)for(auto& e:ed[v])
        if(rem(e)>0 && pi[e.to]>pi[v]+e.cost)
          pi[e.to]=pi[v]+e.cost,ch=1;
      if(!ch)return;
    } // c217eb
    assert(0); // reachable negative cost cycle
  } // f6b5ac
  bool path(int s,int t){
    fill(all(dist),INF); fill(all(ptr),0); dist[s]=0;
    PQ q; q.push({0,s});
    while(!q.empty()){
      int v=q.top().second; q.pop(); ll d=dist[v];
      for(auto& e:ed[v])if(rem(e)>0){
        ll nd=d+e.cost+pi[v]-pi[e.to];
        if(nd>=dist[e.to])continue;
        if(dist[e.to]==INF)its[e.to]=q.push({-nd,e.to});
        else q.modify(its[e.to],{-nd,e.to});
        dist[e.to]=nd;
      } // 239890
    } // 6ae111
    if(dist[t]==INF)return 0;
    rep(v,0,n)if(dist[v]<INF)pi[v]+=dist[v];
    return 1;
  } // 07f946
  ll push(int v,int t,ll f){
    if(v==t)return f;
    on[v]=1; ll ret=0;
    for(int& i=ptr[v];i<sz(ed[v])&&f;i++){
      edge& e=ed[v][i];
      if(on[e.to] || rem(e)<=0 || e.cost+pi[v]-pi[e.to])
          continue;
      ll x=push(e.to,t,min(f,rem(e)));
      e.flow+=x; ed[e.to][e.rev].flow-=x;
      f-=x; ret+=x;
    } // 3c0a54
    return on[v]=0, ret;
  } // 0edbd7
  pair<ll,ll> maxflow(int s,int t){
    ll flow=0,cost=0;
    while(path(s,t))if (ll f=push(s,t,INF))
      flow+=f,cost+=f*(pi[t]-pi[s]);
    return {flow,cost};
  } // a097ae
}; // 20ade0
```

GlobalMinCut.h

Description: Stoer-Wagner global minimum cut of an undirected weighted graph given by its adjacency matrix. Returns cut, one side of the cut. Requires nonnegative weights and n >= 2.

Memory: $\mathcal{O}(N^2)$

Time: $\mathcal{O}(N^3)$

eb8ae6, 34L, 7m

```
pair<ll,vi> globalMinCut(vector<vector<ll>> a) {
  int n=sz(a);
  vi v(n),vis(n),hd(n),tl(n),nx(n,-1); vector<ll> w(n);
  iota(all(v),0); iota(all(hd),0); tl=hd;
  ll best=LLONG_MAX; vi side;
  for(int k=n;k>1;k--) {
    rep(i,0,k) w[v[i]]=vis[v[i]]=0;
    int s = -1, t = -1, pt = -1;
    rep(it,0,k) {
      s = t; t = -1;
      rep(j,0,k) {
        int x=v[j];
        if(!vis[x] && (t<0 || w[x]>w[t]))t=x,pt=j;
      } // a6f180
      if(it==k-1) break;
      vis[t]=1;
      rep(j,0,k) {
        int x=v[j];
        if(!vis[x]) w[x]+=a[t][x];
      } // 63f519
    } // 43521e
    if(w[t]<best) {
      best=w[t]; side.clear();
      for(int x=hd[t];x>=0;x=nx[x]) side.pb(x);
    } // c1172d
    nx[tl[s]]=hd[t]; tl[s]=tl[t];
    rep(j,0,k) {
      int x=v[j];
      if(x!=s && x!=t)a[x][s]=a[s][x]+=a[t][x];
    } // d82ebd
    v[pt]=v[k-1];
  } // 7e3156
  return {best,side};
} // eb8ae6
```

GomoryHu.h

Description: Given a list of edges representing an undirected flow graph, returns edges of the Gomory-Hu tree. The max flow between any pair of vertices is given by minimum edge weight along the Gomory-Hu tree path.

Time: $\mathcal{O}(V)$ Flow Computations

"Dinic.h" dc58a4, 12L, 2m40s

```
typedef array<ll, 3> Edge;
vector<Edge> gomoryHu(int N, vector<Edge> ed) {
  vector<Edge> tree; vi par(N);
  Dinic D(N);
  for(auto [a,b,c]:ed)D.addEdge(a,b,c,c);
  rep(i,1,N){
    tree.pb({i,par[i],D.calc(i,par[i])});
    rep(j,i+1,N)if(par[j]==par[i] && D.leftOfMinCut(j))par[j]=i
        ;
    D.reset();
  } // d16b77
  return tree;
} // 3a874d
```

5.3 Matching

HopcroftKarp.h

Description: Fast bipartite matching algorithm. Graph g should be a list of neighbors of the left partition, and r should be a vector full of -1's of the same size as the right partition. Returns the size of the matching. $r[i]$ will be the match for vertex i on the right side, or -1 if it's not matched.

Time: $\mathcal{O}\left(E\sqrt{V}\right)$ a63a5c, 20L, 5m30s

```
int hopcroftKarp(const vector<vi>& g, vi& r) {
  int n = sz(g), res = 0;
  vi l(n, -1), q(n), d(n);
  auto dfs = [&](auto f, int u) -> bool {
    int t = exchange(d[u], 0) + 1;
    for (int v : g[u])
      if (r[v] == -1 || (d[r[v]] == t && f(f, r[v])))
        return l[u] = v, r[v] = u, 1;
    return 0;
  }; // a95e38
  for (int t = 0, f = 0;; t = f = 0, d.assign(n, 0)) {
    rep(i,0,n) if (l[i] == -1) q[t++] = i, d[i] = 1;
    rep(i,0,t) for (int v : g[q[i]]) {
      if (r[v] == -1) f = 1;
      else if (!d[r[v]]) d[r[v]] = d[q[i]] + 1, q[t++] = r[v];
    } // 64af74
    if (!f) return res;
    rep(i,0,n) if (l[i] == -1) res += dfs(dfs, i);
  } // cdf3b2
} // a63a5c
```

MinimumVertexCover.h

Description: Finds a minimum vertex cover in a bipartite graph. The size is the same as the size of a maximum matching, and the complement is a maximum independent set.

"HopcroftKarp.h" cd4d99, 20L, 5m20s

```
vi cover(const vector<vi>& g, int n, int m) {
  vi match(m, -1);
  int res = hopcroftKarp(g, match);
  vector<bool> lfound(n, true), seen(m);
  for (int it : match) if (it != -1) lfound[it] = false;
  vi q, cover;
  rep(i,0,n) if (lfound[i]) q.push_back(i);
  while (!q.empty()) {
    int i = q.back(); q.pop_back();
    lfound[i] = 1;
    for (int e : g[i]) if (!seen[e] && match[e] != -1) {
      seen[e] = true;
      q.push_back(match[e]);
    } // 46e035
  } // 069994
  rep(i,0,n) if (!lfound[i]) cover.push_back(i);
  rep(i,0,m) if (seen[i]) cover.push_back(n+i);
  assert(sz(cover) == res);
  return cover;
} // cd4d99
```

GeneralMatching.h

Description: Maximum cardinality matching in a general graph. Vertices are 0-indexed. solve() returns the size, get() returns the edges.

Memory: $\mathcal{O}(n+m)$

Time: $\mathcal{O}\left(sqrt(n)mlog_{max(2,1+m/n)}n\right)$ 61c49d, 192L, 46m10s

```
struct GeneralMatching {
  struct Edge { int u,v; };
  static constexpr int INF=1<<30, INNER=-1, FREE=0;
  int n,nh,ql,qr,hs,hl,sp,tim,aug,cc,oid;
  vi off,q,mt,pot,lab,dsu,lh,ln,bh,bn,st;
  vector<Edge> ed,lk,hist;
  GeneralMatching(int N, const vector<Edge>& in):
    n(N),nh(N/2),off(n+2),ed(2*sz(in)) {
    for(auto e:in){
      int a=e.u+1,b=e.v+1;
      off[a+1]++; off[b+1]++;
    } // c69c35
    rep(i,1,n+2)off[i]+=off[i-1];
    for(auto e:in){
      int a=e.u+1,b=e.v+1;
      ed[off[a]++]={a,b};
      ed[off[b]++]={b,a};
    } // 32bc7c
    for(int i=n+1;i;i--)off[i]=off[i-1];
    off[0]=0;
  } // d54fef
  int find(int x){return dsu[x]==x?x:dsu[x]=find(dsu[x]);}
  void push(vi& h,vi& nx,int a,int b){nx[b]=h[a]; h[a]=b;}
  void reset(){tim=0; aug=INF; cc=0; oid=1; hs=hl=0;}
  void clear(){
    ql=qr=0; rep(i,tim,nh+1)lh[i]=-1;
    rep(i,1,n+1)pot[i]=1,dsu[i]=i,bh[i]=-1;
  } // db956e
  void grow(int x,int y,int z){
    lab[y]=INNER; pot[y]=tim; lk[z]={x,y};
    lab[z]=lab[x]; pot[z]=tim+1; q[qr++]=z;
  } // 103420
  void contract(int x,int y){
    int a=find(x),b=find(y),h=-(++cc)+INNER,lca=-1;
    lab[mt[a]]=lab[mt[b]]=h;
    for(;;){
      if(mt[b])swap(a,b);
      a=lca=find(lk[a].u);
      if(lab[mt[a]]==h)break;
      lab[mt[a]]=h;
    } // 9dbcdd
    for(int v:{dsu[x],dsu[y]})
      for(;v!=lca;v=dsu[lk[v].u]){
        int w=mt[v];
        lk[w]={x,y}; lab[w]=lab[x];
        pot[w]=1+(tim-pot[w])+tim;
        q[qr++]=w;
        dsu[v]=dsu[w]=lca;
        hist[hl++]={v,lca};
        hist[hl++]={w,lca};
      } // ccbc18
  } // 95e631
  bool search(){
    while(ql<qr){
      int x=q[ql++],lx=lab[x],px=pot[x],bx=find(x);
      for(int i=off[x];i<off[x+1];i++){
        int y=ed[i].v;
        if(lab[y]>0){
          int nt=(px+pot[y])>>1;
          if(lx!=lab[y]){
            if(nt==tim)return 1;
            aug=min(aug,nt);
          } // d85bff
          else{
            if(bx==find(y))continue;
            if(nt==tim)contract(x,y),bx=find(x);
            else if(nt<=nh)push(lh,ln,nt,i);
          } // f53f25
        } // 5fff25
        else if(lab[y]==FREE){
          int nt=px+1;
          if(nt==tim)grow(x,y,mt[y]);
          else if(nt<=nh)push(lh,ln,nt,i);
        } // 122893
      } // 1279b5
    } // 8b6505
    return 0;
  } // 2184a7
  bool advance(){
    int lim=min(nh+1,aug);
    for(++tim;tim<=lim;tim++){
      hs=hl;
      if(tim==lim)break;
      bool upd=0;
      for(int i=lh[tim];i>=0;i=ln[i]){
        auto e=ed[i]; int x=e.u,y=e.v;
        if(lab[y]>0){
          if(pot[x]+pot[y]!=(tim<<1) || find(x)==find(y))
            continue;
          if(lab[x]!=lab[y]) return aug=tim,0;
          contract(x,y); upd=1;
        } // 2545e0
        else if(lab[y]==FREE) grow(x,y,mt[y]),upd=1;
      } // 2af36c
      lh[tim]=-1;
      if(upd)return 0;
    } // 5c6ff1
    return tim>nh;
  } // 1c3d31
  bool phase(){
    lab[0]=FREE;
    rep(i,1,n+1)
      if(!mt[i])q[qr++]=i,lab[i]=i;
      else lab[i]=FREE;
    for(;;){
      if(search())break;
      if(advance())return 0;
      if(tim==aug)break;
    } // b079de
    rep(i,1,n+1)
      if(lab[i]>0)pot[i]-=tim;
      else if(lab[i]<0)pot[i]=1+tim-pot[i];
    return 1;
  } // c214d6
  void rematch(int v,int w){
    int x=mt[v]; mt[v]=w;
    if(mt[x]!=v)return;
    if(lk[v].v==find(lk[v].v))mt[x]=lk[v].u,rematch(mt[x],x);
    else{
      int a=lk[v].u,b=lk[v].v;
      rematch(a,b); rematch(b,a);
    } // f95725
  } // 08f0ab
  bool dfs(int x,int bx){
    int px=pot[x],lx=lab[bx];
    for(int i=off[x];i<off[x+1];i++){
      int y=ed[i].v;
      if(px+pot[y])continue;
      int by=find(y),ly=lab[by];
      if(ly>0){
        if(lx>=ly)continue;
        int beg=sp;
        for(int v=by;v!=bx;v=find(lk[v].u)){
          int w=find(mt[v]);
          st[sp++]=w;
          lk[w]={x,y};
          dsu[v]=dsu[w]=bx;
        } // bec1fd
        while(sp>beg){
          int v=st[--sp];
          for(int u=bh[v];u>=0;u=bn[u]) if(dfs(u,bx))return sp=
              beg,1;
        } // d1fbd2
      } // 7376f9
      else if(ly==FREE){
        lab[by]=INNER;
```

```
      int z=mt[by];
      if(!z){
        rematch(x,y); rematch(y,x);
        return 1;
      } // 8efdab
      int bz=find(z);
      lk[bz]={x,y}; lab[bz]=oid++;
      for(int v=bh[bz];v>=0;v=bn[v])if(dfs(v,bz))return 1;
    } // 97f337
  } // 2ccccc
  return 0;
} // 2c34bf
int augment(){
  rep(i,1,n+1)dsu[i]=i;
  rep(i,0,hs)dsu[hist[i].u]=hist[i].v;
  rep(i,1,n+1)lab[i]=FREE,push(bh,bn,find(i),i);
  int ret=0;
  rep(i,1,n+1)if(!mt[i]){
    int b=dsu[i];
    if(lab[b]!=FREE)continue;
    lab[b]=oid++;
    for(int v=bh[b];v>=0;v=bn[v])if(dfs(v,b)){ret++;break;}
  } // 7aff9b
  assert(ret); return ret;
} // 017319
void init(){
  q.resize(n); mt.assign(n+1,0);
  pot.assign(n+1,1); lab.assign(n+1,FREE);
  lk.assign(n+1,{0,0}); hist.resize(n);
  dsu.resize(n+1); lh.assign(nh+1,-1);
  ln.resize(sz(ed)); bh.assign(n+1,-1);
  bn.resize(n+1); st.resize(n); ql=qr=sp=0;
} // 0c3978
int solve(){
  init(); int ans=0;
  while(2*ans+1<n){
    reset();
    if(!phase())break;
    ans+=augment(); clear();
  } // c6f51c
  return ans;
} // 806d68
vector<Edge> get(){
  vector<Edge> ans;
  rep(i,1,n+1)if(mt[i]>i)ans.pb({i-1,mt[i]-1});
  return ans;
} // dbe34c
}; // 61c49d
```

IncrementalMatching.h

Description: Incremental bipartite matching. Left vertices are activated ONCE with activate(a); returns whether the maximum matching increases. All edges of a must exist before it is activated.
Memory: $\mathcal{O}(L+R+E)$.
Time: $\mathcal{O}(E)$ worst case per activation, usually much faster

f07a46, 20L, 4m50s

```
struct IncrementalMatching {
  vi st,mt,d; vector<vi> g; int t=0;
  IncrementalMatching(int n, int m):st(n),mt(m,-1),d(n),g(n){}
  void add(int a,int b){g[a].pb(b);}
  bool dfs(int a){
    st[a]=-t;
    for(int b:g[a]){
      int c=mt[b]; if(c<0)return mt[b]=a,1;
      int nd=d[a]+1;
      if(st[c]!=t && st[c]!=-t)d[c]=nd,st[c]=t;
      else if(nd<d[c])d[c]=nd;
    } // 9aee70
    for(int b:g[a]){
      int c=mt[b];
      if(d[c]==d[a]+1 && st[c]!=-t && dfs(c))return mt[b]=a,1;
    } // c01e8b
    return 0;
  } // 2f0bdf
  bool activate(int a){st[a]=++t,d[a]=0;return dfs(a);}
}; // f07a46
```

5.4 Connectivity

SCC.h

Description: Finds strongly connected components in a directed graph. If vertices u, v belong to the same component, we can reach u from v and vice versa.
Usage: scc(graph, [&](vi& v) { ... }) visits all components in reverse topological order. comp[i] holds the component index of a node (a component only has edges to components with lower index). ncomps will contain the number of components.
Time: $\mathcal{O}(E+V)$

76b5c9, 24L, 5m20s

```
vi val, comp, z, cont;
int Time, ncomps;
template<class G, class F> int dfs(int j, G& g, F& f) {
  int low = val[j] = ++Time, x; z.push_back(j);
  for (auto e : g[j]) if (comp[e] < 0)
    low = min(low, val[e] ?: dfs(e,g,f));

  if (low == val[j]) {
    do {
      x = z.back(); z.pop_back();
      comp[x] = ncomps;
      cont.push_back(x);
    } while (x != j); // ae85bd
    f(cont); cont.clear();
    ncomps++;
  } // 64c1b9
  return val[j] = low;
} // 3513bd
template<class G, class F> void scc(G& g, F f) {
  int n = sz(g);
  val.assign(n, 0); comp.assign(n, -1);
  Time = ncomps = 0;
  rep(i,0,n) if (comp[i] < 0) dfs(i, g, f);
} // 56b050
```

TwoEdgeCC.h

Description: Finds all two edge connected components in an undirected graph (comp). Edge u-v is a bridge if comp[u] != comp[v]. Can return the tree.
Time: $\mathcal{O}(E+V)$

ff5fea, 29L, 6m30s

```
struct TwoEdgeCC {
  const vector<vi>& g;
  const vector<pii>& ed;
  vi tin,st,comp;
  int tim=0,cc=0;
  TwoEdgeCC(const vector<vi>& G,const vector<pii>& E):
    g(G),ed(E),tin(sz(G)),comp(sz(G)){
    rep(v,0,sz(g))if(!tin[v])dfs(v,-1);
  } // 616c71
  int dfs(int v,int pe){
    int low=tin[v]=++tim,si=sz(st); st.pb(v);
    for(int e:g[v])if(e!=pe){
      auto [a,b]=ed[e]; int u=a^b^v;
      if(tin[u])low=min(low,tin[u]);
      else low=min(low,dfs(u,e));
    } // 4edd08
    if(low==tin[v]){
      while(sz(st)>si)comp[st.back()]=cc,st.pop_back();
      cc++;
    } // b08279
    return low;
  } // 452901
  vector<vi> tree() const {
    vector<vi> t(cc);
    for(auto [u,v]:ed)if(comp[u]!=comp[v])
      t[comp[u]].pb(comp[v]),t[comp[v]].pb(comp[u]);
    return t;
  } // e88f5d
}; // ff5fea
```

TwoVertexCC.h

Description: Vertex-biconnected components of an undirected graph. comp[e] is the component of edge e; art[v] tells if v is an articulation point. Bridges form singleton-edge components. Handles parallel edges; assumes no self-loops.
Memory: $\mathcal{O}(N+M)$
Time: $\mathcal{O}(N+M)$

12035f, 53L, 11m50s

```
struct TwoVertexCC {
  const vector<vi>& g;
  const vector<pii>& ed;
  vi tin,st,comp;
  vector<char> art;
  int tim=0,cc=0;
  TwoVertexCC(const vector<vi>& G,const vector<pii>& E):
    g(G),ed(E),tin(sz(G)),comp(sz(E),-1),art(sz(G)){
    st.reserve(sz(E));
    rep(v,0,sz(g))if(!tin[v])dfs(v,-1);
  } // 11442f
  int dfs(int v,int pe){
    int low=tin[v]=++tim,ch=0;
    for(int e:g[v])if(e!=pe){
      auto [a,b]=ed[e]; int u=a^b^v;
      if(tin[u]){
        if(tin[u]<tin[v])st.pb(e),low=min(low,tin[u]);
      } // 4c4942
      else{
        ch++; st.pb(e);
        int up=dfs(u,e); low=min(low,up);
        if(up>=tin[v]){
          if(pe!=-1)art[v]=1;
          for(;;){
            int x=st.back(); st.pop_back();
            comp[x]=cc;
            if(x==e)break;
          } // 085e8b
          cc++;
        } // 0c6742
      } // 7a948d
    } // 8e53fa
    if(pe==-1)art[v]=ch>1;
    return low;
  } // d15668
  pair<vector<vi>,vi> blockCut() const {
    int n=sz(g),k=cc;
    vi id(n,-1),seen(cc,-1); //-1 if isolated
    rep(v,0,n)
      if(art[v])id[v]=k++; // articulation points' id is their
          separate component
      else if(sz(g[v]))id[v]=comp[g[v][0]];
    vector<vi> t(k);
    rep(v,0,n)if(art[v])for(int e:g[v]){
      int c=comp[e];
      if(seen[c]!=v){
        seen[c]=v;
        t[id[v]].pb(c);
        t[c].pb(id[v]);
```

```
    } // d5d12e
  } // 47ceca
  return {t,id}; // tree of components, mapping from node to
      id in tree
 } // 08eda5
}; // 12035f
```

2sat.h

Description: Calculates a valid assignment to boolean variables a, b, c,... to a 2-SAT problem, so that an expression of the type $(a||b)\&\&(!a||c)\&\&(d||!b)\&\&...$ becomes true, or reports that it is unsatisfiable. Negated variables are represented by bit-inversions (~x).
Usage: `TwoSat ts(number of boolean variables);`
`ts.either(0, ~3); // Var 0 is true or var 3 is false`
`ts.setValue(2); // Var 2 is true`
`ts.atMostOne({0,~1,2}); // <= 1 of vars 0, ~1 and 2 are true`
`ts.solve(); // Returns true iff it is solvable`
`ts.values[0..N-1] holds the assigned values to the vars`
Time: $\mathcal{O}(N+E)$, where N is the number of boolean variables, and E is the number of clauses. 8e1208, 48L, 11m

```
struct TwoSat {
 int N; vector<vi> gr; vi values; // 0 = false, 1 = true
 TwoSat(int n = 0) : N(n), gr(2*n) {}
 void either(int f, int j) {
  f = max(2*f, -1-2*f);
  j = max(2*j, -1-2*j);
  gr[f].push_back(j^1);
  gr[j].push_back(f^1);
 } // 516db0
 void setValue(int x) { either(x, x); }
  int addVar() { // (optional)
  gr.emplace_back();
  gr.emplace_back();
  return N++;
 } // 7b5f84
 void atMostOne(const vi& li) { // (optional)
  if (sz(li) <= 1) return;
  int cur = ~li[0];
  rep(i,2,sz(li)) {
   int next = addVar();
   either(cur, ~li[i]);
   either(cur, next);
   either(~li[i], next);
   cur = ~next;
  } // 8d3782
  either(cur, ~li[1]);
 } // 10f2ea
 vi val, comp, z; int time = 0;
 int dfs(int i) {
  int low = val[i] = ++time, x; z.push_back(i);
  for(int e : gr[i]) if (!comp[e])
   low = min(low, val[e] ?: dfs(e));
  if (low == val[i]) do {
   x = z.back(); z.pop_back();
   comp[x] = low;
   if (values[x>>1] == -1)
    values[x>>1] = x&1;
  } while (x != i); // b15351
  return val[i] = low;
 } // ef583a
 bool solve() {
  values.assign(N, -1);
  val.assign(2*N, 0); comp = val;
  rep(i,0,2*N) if (!comp[i]) dfs(i);
  rep(i,0,N) if (comp[2*i] == comp[2*i+1]) return 0;
  return 1;
 } // 2bb76d
}; // 8e1208
```

DominatorTree.h

Description: Immediate dominators from root. idom[root]=root and idom[v]=-1 if v is unreachable. tree() builds the dominator tree on demand.
Memory: $\mathcal{O}(N+M)$
Time: $\mathcal{O}((N+M)logN)$ fa23b2, 58L, 15m40s

```
struct DominatorTree {
 vi idom;
 DominatorTree(const vector<vi>& g, int root) {
  int n=sz(g); vi id(n,-1), rev, par;
  { // DFS
   vector<pii> st={{root,0}};
   id[root]=0; rev.pb(root); par.pb(-1);
   while(sz(st)) {
    auto &[u,i]=st.back();
    if(i==sz(g[u])) { st.pop_back(); continue; }
    int v=g[u][i++];
    if(id[v]<0)id[v]=sz(rev),rev.pb(v),par.pb(id[u]),st.pb(
        {v,0});
   } // d0465c
  } // 694b7a
  int N=sz(rev); vi off(N+1);
  rep(i,0,N)for(int v : g[rev[i]])off[id[v]+1]++;
  rep(i,1,N+1)off[i]+=off[i-1];
  vi pre(off[N]);
  rep(i,0,N)for(int v : g[rev[i]])pre[off[id[v]]++]=i;
  for(int i=N;i;i--)off[i]=off[i-1];
  off[0]=0;
  fill(all(id),-1); // reused as bucket heads
  vi sd(N),dom(N,-1),dsu(N),lab(N),st;
  iota(all(sd),0); iota(all(dsu),0); iota(all(lab),0);
  st.reserve(N);
  auto fnd = [&](int v) {
   if(dsu[v]==v) return v;
   st.clear(); int x=v;
   while(dsu[x]!=x) st.pb(x),x=dsu[x];
   int a=st.back();
   for(int k=sz(st)-1;k--;) {
    int u=st[k],p=dsu[u];
    if(sd[lab[p]]<sd[lab[u]]) lab[u]=lab[p];
    dsu[u]=a;
   } // e8cc0f
   return lab[v];
  }; // 9fda8c
  for(int u=N;u--;) {
   rep(e,off[u],off[u+1])sd[u]=min(sd[u],sd[fnd(pre[e])]);
   if(u) dom[u]=id[sd[u]],id[sd[u]]=u;
   for(int v=id[u];v>=0;) {
    int nx=dom[v],w=fnd(v);
    dom[v]=sd[w]==sd[v] ? sd[v] : w;
    v=nx;
   } // 883b39
   if(u) dsu[u]=par[u];
  } // 64776c
  rep(u,1,N)if(dom[u]!=sd[u])dom[u]=dom[dom[u]];
  fill(all(id),-1); id[root]=root;
  rep(u,1,N) id[rev[u]]=rev[dom[u]];
  idom=move(id);
 } // 27c9d2
 vector<vi> tree() const {
  vector<vi> t(sz(idom));
  rep(v,0,sz(idom))if(idom[v]>=0 && idom[v]!=v)t[idom[v]].pb(
      v);
  return t;
 } // 34d69a
}; // fa23b2
```

5.5 Cliques

MaximalCliques.h

Description: Runs a callback for all maximal cliques in a graph (given as a symmetric bitset matrix; self-edges not allowed). Callback is given a bitset representing the maximal clique.
Time: $\mathcal{O}\left(3^{n/3}\right)$, much faster for sparse graphs b0d5b1, 12L, 3m

```
typedef bitset<128> B;
template<class F>
void cliques(vector<B>& eds, F f, B P = ~B(), B X={}, B R={}) {
 if (!P.any()) { if (!X.any()) f(R); return; }
 auto q = (P | X)._Find_first();
 auto cands = P & ~eds[q];
 rep(i,0,sz(eds)) if (cands[i]) {
  R[i] = 1;
  cliques(eds, f, P & eds[i], X & eds[i], R);
  R[i] = P[i] = 0; X[i] = 1;
 } // 181f8f
} // c9dc5f
```

MaximumClique.h

Description: Quickly finds a maximum clique of a graph (given as symmetric bitset matrix; self-edges not allowed). Can be used to find a maximum independent set by finding a clique of the complement graph.
Time: Runs in about 1s for n=155 and worst case random graphs (p=.90). Runs faster for sparse graphs. f7c0bc, 49L, 14m

```
typedef vector<bitset<200>> vb;
struct Maxclique {
 double limit=0.025, pk=0;
 struct Vertex { int i, d=0; };
 typedef vector<Vertex> vv;
 vb e;
 vv V;
 vector<vi> C;
 vi qmax, q, S, old;
 void init(vv& r) {
  for (auto& v : r) v.d = 0;
  for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
  sort(all(r), [](auto a, auto b) { return a.d > b.d; });
  int mxD = r[0].d;
  rep(i,0,sz(r)) r[i].d = min(i, mxD) + 1;
 } // 7c428e
 void expand(vv& R, int lev = 1) {
  S[lev] += S[lev - 1] - old[lev];
  old[lev] = S[lev - 1];
  while (sz(R)) {
   if (sz(q) + R.back().d <= sz(qmax)) return;
   q.push_back(R.back().i);
   vv T;
   for(auto v:R) if (e[R.back().i][v.i]) T.push_back({v.i});
   if (sz(T)) {
    if (S[lev]++ / ++pk < limit) init(T);
    int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1, 1);
    C[1].clear(), C[2].clear();
    for (auto v : T) {
     int k = 1;
     auto f = [&](int i) { return e[v.i][i]; };
     while (any_of(all(C[k]), f)) k++;
     if (k > mxk) mxk = k, C[mxk + 1].clear();
     if (k < mnk) T[j++].i = v.i;
     C[k].push_back(v.i);
    } // 3221ac
    if (j > 0) T[j - 1].d = 0;
    rep(k,mnk,mxk + 1) for (int i : C[k])
     T[j].i = i, T[j++].d = k;
    expand(T, lev + 1);
   } else if (sz(q) > sz(qmax)) qmax = q; // 2a0537
   q.pop_back(), R.pop_back();
  } // 87639b
```

```
  } // f0a49d
  vi maxClique() { init(V), expand(V); return qmax; }
  Maxclique(vb conn) : e(conn), C(sz(e)+1), S(sz(C)), old(S) {
    rep(i,0,sz(e)) V.push_back({i});
  } // 36accb
}; // b63641
```

5.6 Trees

5.6.1 Algorithms

CompressTree.h

Description: Given a rooted tree and a subset S of nodes, compute the minimal subtree that contains all the nodes by adding all (at most $|S|-1$) pairwise LCA's and compressing edges. S must be non-empty. Returns a list of (par, orig_index) representing a tree rooted at 0. The root points to itself.
Time: $\mathcal{O}(|S|\log|S|)$

7347c3, 26L, 6m

```
struct LCA { // replace in contest!
  vi time;
  int lca(int a, int b) { return -1; }
}; // 32d408

typedef vector<pair<int, int>> vpi;
vpi compressTree(LCA& lca, const vi& subset) {
  static vi rev; rev.resize(sz(lca.time));
  vi li = subset, &T = lca.time;
  auto cmp = [&](int a, int b) { return T[a] < T[b]; };
  sort(all(li), cmp);
  int m = sz(li)-1;
  rep(i,0,m) {
    int a = li[i], b = li[i+1];
    li.push_back(lca.lca(a, b));
  } // 677c62
  sort(all(li), cmp);
  li.erase(unique(all(li)), li.end());
  rep(i,0,sz(li)) rev[li[i]] = i;
  vpi ret = {pii(0, li[0])};
  rep(i,0,sz(li)-1) {
    int a = li[i], b = li[i+1];
    ret.emplace_back(rev[lca.lca(a, b)], b);
  } // 5efe90
  return ret;
} // 83c9a2
```

SmallToLarge.h

Description: Small-to-large. While out(v) is called, the maintained state contains exactly the subtree of v. add(v) inserts v and del(v) undoes it. Root is configurable.
Memory: $\mathcal{O}(N)$.
Time: $\mathcal{O}(NlogN * (add + del) + N * (out))$.

dbb339, 34L, 10m

```
template<class A,class D,class Q>
void smallToLarge(const vector<vi>& g, A add, D del, Q out, int
    r){
  int n=sz(g);
  vi sub(n, 1), pos(n, -1), pre, st={r};
  pre.reserve(n); st.reserve(n); pos[r]=r; // pos starts as
      parent vector
  while(sz(st)){
    int v = st.back(); st.pop_back(); pre.pb(v);
    for(int f : g[v])if (f != pos[v])pos[f] = v, st.pb(f);
  } // 4998a5
  for(int i = n; i-->1; sub[pos[pre[i]]] += sub[pre[i]]);
  rep(i,0,n)pos[pre[i]] = i;
  auto heavy = [&](int v){
    int b = -1;
    for(int f : g[v])if (pos[f] > pos[v] && (b < 0 || sub[f] >
        sub[b]))b=f;
    return b;
  }; // c075d2
  st.clear(); st.pb(2*r+1); // 2*v + keep, ~ if out
  while(sz(st)){
    int z = st.back(); st.pop_back();
    if(z >= 0){
      int v = z/2, b = heavy(v); st.pb(~z);
      if(b >= 0)st.pb(2*b+1);
      for(int f : g[v])if (pos[f] > pos[v] && f != b)st.pb(2*f)
          ;
    } // b94b34
    else{
      z=~z; int v=z/2, keep = z%2, b = heavy(v);
      add(v);
      for(int f : g[v])if (pos[f] > pos[v] && f != b)
        rep(i,pos[f],pos[f]+sub[f])add(pre[i]);
      out(v);
      if(!keep)rep(i,pos[v],pos[v]+sub[v])del(pre[i]);
    } // 8c11b2
  } // 5ca0a8
} // b5f41b
```

CentroidDecomp.h

Description: Centroid decomposition. process(c,p,dead) is called before removing centroid c; p is its parent in the centroid tree. Use const auto & dead.
Memory: $\mathcal{O}(N)$.
Time: $\mathcal{O}(N\log N + processcalls)$.

c780f6, 26L, 6m50s

```
template<class F>
void centroidDecomp(const vector<vi>& g, F process, int root){
  int n=sz(g); vi sub(n); vector<char> dead(n);
  vector<pii> ord; ord.reserve(n);
  auto dfs=[&](auto&& go, int r, int cp)->void {
    ord.clear(); ord.pb({r,-1});
    rep(i,0,sz(ord)){
      auto [v, p] = ord[i]; sub[v] = 1;
      for(int f : g[v])if (f != p && !dead[f])ord.pb({f,v});
    } // ef3ae8
    for(int i=sz(ord);i--;){
      auto [v, p] = ord[i];
      if(p >= 0)sub[p] += sub[v];
    } // c3dca1
    int s = sz(ord), c = r, p = -1;
    for(;;){
      int nx=-1;
      for(int f : g[c]) if(f != p && !dead[f] && 2*sub[f] > s){
          nx=f; break;}
      if(nx < 0)break;
      p=c; c=nx;
    } // ef7142
    process(c, cp, dead); dead[c] = 1;
    for(int f : g[c])if (!dead[f]) go(go, f, c);
  }; // f0008c
  dfs(dfs, root, -1);
} // 03db34
```

5.6.2 Structures

HLD.h

Description: HLD with lazy path updates and queries. EDGE determines whether values are stored on edges; edge values are stored at the child. Root (r) is configurable. query(u,v) follows the order from u to v. Can be adapted to use normal SegTree (may double performance) S requires T,L,id,op,ch,cmp,rev.
Memory: $\mathcal{O}(N)$.
Time: $\mathcal{O}\left(log^2 N\right)$ per path operation, $\mathcal{O}(logN)$ per point operation.

"../data-structures/LazySegmentTree.h" f84b2b, 58L, 17m30s

```
template<class S, bool EDGE=false> struct HLD {
  using T=typename S::T; using L=typename S::L;
  int n; vi p,siz,hd,ti,hv; SegBeats<S> seg;
  HLD(const vector<vi>& g, int r):n(sz(g)),
    p(n,-1),siz(n,1),hd(n),ti(n),hv(n,-1),seg(n) {
    p[r]=r; vi ord={r}; ord.reserve(n);
    rep(i,0,sz(ord)) for(int f : g[ord[i]])
      if (f != p[ord[i]])p[f]=ord[i], ord.pb(f);
    reverse(all(ord));
    for(int v : ord) for(int f : g[v]) if (p[f] == v){
      siz[v] += siz[f];
      if (hv[v] < 0 || siz[f] > siz[hv[v]]) hv[v] = f;
    } // 64260e
    ord.clear(); ord.pb(r); int tim=0;
    while(sz(ord)) {
      int h = ord.back(); ord.pop_back();
      for(int v = h; v >= 0; v = hv[v]) {
        hd[v] = h; ti[v]=tim++;
        for(int f : g[v]) if(p[f]==v && f != hv[v])ord.pb(f);
      } // 60efcc
    } // 572ae1
  } // d9b4c8
  template<class A, class B> void process(int u, int v, A up, B
      down){
    while(hd[u] != hd[v])
      if(ti[hd[u]] > ti[hd[v]])up(ti[hd[u]],ti[u]),u=p[hd[u]];
      else down(ti[hd[v]],ti[v]),v=p[hd[v]];
    if(ti[u] > ti[v])up(ti[v]+EDGE, ti[u]);
    else  down(ti[u]+EDGE, ti[v]); // may call seg with l > r !
  } // 6ff1b1
  T query(int u, int v) {
    T a=S::id,b=S::id; process(u, v,
      [&](int l,int r){a=S::op(a,S::rev(seg.query(l,r)));},
      [&](int l,int r){b=S::op(seg.query(l,r),b);});
    return S::op(a,b);
  } // a5f442
  void update(L x, int u, int v) {
    auto f=[&](int l,int r){seg.update(x,l,r);};
    process(u,v,f,f);
  } // 0182c8
}; // f84b2b

/* Additional operations:
  void update(L x,int v) {seg.update(x,ti[v],ti[v]);}
  int lca(int u,int v) {
    while(hd[u]!=hd[v])
      if(ti[hd[u]]>ti[hd[v]]) u=p[hd[u]];
      else v=p[hd[v]];
    return ti[u]<ti[v] ? u : v;
  } // 521f41
  T querySubtree(int v) {
    int l=ti[v]+EDGE,r=ti[v]+siz[v]-1;
    return l>r ? S::id : seg.query(l,r);
  } // 36a519
  void updateSubtree(L x,int v) {
    int l=ti[v]+EDGE,r=ti[v]+siz[v]-1;
    if(l<=r) seg.update(x,l,r);
  } // 4a3d3b
*/
```

LinkCutTree.h

Description: Link-Cut Tree for a dynamic forest. Supports link/cut, connectivity, point set and path queries. Nodes are internally represented as 1-based, API is 0-based. S requires T, id, op and rev. query(u,v) follows path order u to v. Store edge values by creating an extra node for each edge.
Memory: $\mathcal{O}(N)$.
Time: $\mathcal{O}(\log N)$ amortized per operation.

cfccbc, 73L, 18m40s

```
template<class S> struct LCT {
  using T=typename S::T;
  struct N {
    int p=0,c[2]={}; T v=S::id,ac=S::id; bool rev=0;
    N(T V=S::id):v(V),ac(V){}
  }; // 262966
  vector<N> t; vi st;
  LCT(int n=0):t(n+1){st.reserve(n+1);}
  LCT(vector<T> a):t(1){
    for(T v:a)t.eb(v);
    st.reserve(sz(t));
  } // c0ae74
  int node(T v=S::id){return t.eb(v), sz(t)-2;} // returns 0-based id
  bool root(int x){
    int p=t[x].p;
    return !p || (t[p].c[0]!=x && t[p].c[1]!=x);
  } // becbbc
  void pull(int x){
    t[x].ac=S::op(S::op(t[t[x].c[0]].ac,t[x].v), t[t[x].c[1]].ac);
  } // 74d492
  void push(int x){
    if(t[x].rev) flip(t[x].c[0]),flip(t[x].c[1]),t[x].rev=0;
  } // a7444f
  void flip(int x){
    if(!x)return;
    swap(t[x].c[0],t[x].c[1]);
    t[x].ac=S::rev(t[x].ac); t[x].rev^=1;
  } // c22361
  void rot(int x){
    int p=t[x].p,g=t[p].p,d=t[p].c[1]==x,b=t[x].c[d^1];
    if(!root(p))t[g].c[t[g].c[1]==p]=x;
    t[x].p=g; t[x].c[d^1]=p; t[p].p=x;
    t[p].c[d]=b; if(b)t[b].p=p;
    pull(p); pull(x);
  } // 5b39cb
  void splay(int x){
    int y=x; st.clear(); st.pb(y);
    while(!root(y))st.pb(y=t[y].p);
    while(sz(st))push(st.back()),st.pop_back();
    while(!root(x)){
      int p=t[x].p,g=t[p].p;
      if(!root(p))rot((t[p].c[0]==x)==(t[g].c[0]==p)?p:x);
      rot(x);
    } // 718b95
  } // 45b79e
  int access(int x){
    int last=0;
    for(int y=x;y;y=t[y].p){
      splay(y); t[y].c[1]=last;
      pull(y); last=y;
    } // e93ec1
    splay(x); return last;
  } // 5468e0
  void rootify(int x){ access(x); flip(x); }
  int findRoot(int x){
    access(x); while(push(x),t[x].c[0])x=t[x].c[0];
    splay(x); return x;
  } // 3f7c70
  bool connected(int a,int b){ a++; b++; return a==b || findRoot(a)==findRoot(b); }
  void link(int a,int b){
    a++; b++; rootify(a);
    assert(findRoot(b)!=a); t[a].p=b;
  } // 5b8006
  void cut(int a,int b){
    a++; b++; rootify(a); access(b);
    assert(t[b].c[0]==a && !t[a].c[1]);
    t[b].c[0]=t[a].p=0; pull(b);
  } // 0f4217
  void set(int x,T v){ x++; access(x);t[x].v=v; pull(x); }
  T query(int a,int b){
    a++; b++; rootify(a); access(b); return t[b].ac;
  } // 3f5e61
}; // cfccbc
```

TopTreeDiameter.h

Description: Dynamic forest diameter with nonnegative node weights. Vertices are 0-indexed. Use an extra node of weight w for each edge. C is the maximum number of real nodes (vertices + edge nodes).
Memory: $\mathcal{O}(C)$.
Time: $\mathcal{O}(\log N)$ amortized.

0a2dc1, 143L, 34m30s

```
template<class T> struct TopTreeDiameter {
  struct D { T s=0,l=0,r=0,d=0; };
  struct V { T a=0,b=0,d=0; };
  static D cat(D x,D y){return {
    x.s+y.s,
    max(x.l,x.s+y.l),
    max(y.r,y.s+x.r),
    max({x.d,y.d,x.r+y.l})
  };} // 78983b // 78983b
  static void ins(V& z,T x){
    if(x>z.a)z.b=z.a,z.a=x;
    else if(x>z.b)z.b=x;
  } // 699667
  static V rake(V x,V y){
    V z; z.d=max(x.d,y.d);
    ins(z,x.a); ins(z,x.b);
    ins(z,y.a); ins(z,y.b);
    return z;
  } // af2be2
  static V virt(D x){return {x.l,0,x.d};}
  static D one(T w,V v){return {w,w+v.a,w+v.a,max(v.d,w+v.a+v.b)};}
  struct N {
    int p=-1,c[4]={-1,-1,-1,-1};
    T w=0; D d; V v;
    bool rev=0,fake=1;
    N(){}
    N(T W):w(W),d(one(W,{})),fake(0){}
  }; // 2a09f8
  vector<N> t; vi pool, st;
  int cap,nxt;
  TopTreeDiameter(int n,int C):t(2*C),cap(C),nxt(n){
    rep(i,0,n)t[i]=N(0);
    rep(i,C,2*C)pool.pb(i);
    st.reserve(2*C);
  } // 1d1ebf
  int node(T w){return t[nxt]=N(w), nxt++;}
  D gd(int x){return x<0?D():t[x].d;}
  V gv(int x){return x<0?V():(t[x].fake?t[x].v:virt(t[x].d));}
  void pull(int x){
    if(t[x].fake)t[x].v=rake(gv(t[x].c[2]),gv(t[x].c[3]));
    else {
      V v=rake(gv(t[x].c[2]),gv(t[x].c[3]));
      t[x].d=cat(cat(gd(t[x].c[0]),one(t[x].w,v)),gd(t[x].c[1]));
    } // af4bfb
  } // 9f7f92
  void flip(int x){
    if(x<0||t[x].fake)return;
    swap(t[x].c[0],t[x].c[1]);
    swap(t[x].d.l,t[x].d.r);
    t[x].rev^=1;
  } // d4377e
  void push(int x){
    if(x>=0&&t[x].rev)flip(t[x].c[0]),flip(t[x].c[1]),t[x].rev=0;
  } // 6b1995
  int dir(int x){
    int p=t[x].p;
    if(p<0)return -1;
    rep(i,0,4)if(t[p].c[i]==x)return i;
    return -1;
  } // 6c5574
  bool root(int x){
    int p=t[x].p;
    return p<0||(t[x].fake?!t[p].fake:dir(x)>=2);
  } // 34fed4
  void attach(int x,int d,int y){
    t[x].c[d]=y;
    if(y>=0)t[y].p=x;
    pull(x);
  } // 05238a
  void rot(int x){
    int p=t[x].p,g=t[p].p,dx=dir(x),dp=dir(p);
    attach(p,dx,t[x].c[dx^1]); attach(x,dx^1,p);
    if(dp>=0)attach(g,dp,x);
    else t[x].p=g;
  } // 2cb71c
  void splay(int x){
    int y=x; st.clear(); st.pb(y);
    while(!root(y))st.pb(y=t[y].p);
    while(sz(st))push(st.back()),st.pop_back();
    while(!root(x)){
      int p=t[x].p;
      if(!root(p))rot(dir(x)==dir(p)?p:x);
      rot(x);
    } // a5e18c
  } // 5ad1e5
  int fake(){
    assert(sz(pool));
    int x=pool.back();
    pool.pop_back();
    t[x]=N(); return x;
  } // 9f13db
  void addVirt(int x,int y){
    if(y<0)return;
    rep(d,2,4)if(t[x].c[d]<0)return attach(x,d,y);
    int z=fake(); attach(z,2,t[x].c[2]);
    attach(z,3,y); attach(x,2,z);
  } // 09690d
  void remVirt(int x){
    int p=t[x].p;
    if(t[p].fake){
      int g=t[p].p,o=t[p].c[dir(x)^1],d=dir(p);
      attach(g,d,o);
      if(t[g].fake)splay(g);
      pool.pb(p);
    } // e63046
    else attach(p,dir(x),-1);
    t[x].p=-1;
  } // 9966f7
  int realPar(int x){
    int p=t[x].p;
    if(!t[p].fake)return p;
    splay(p); return t[p].p;
  } // a07057
```

```
  int access(int x){
    pull(x); splay(x); addVirt(x,t[x].c[1]);
    attach(x,1,-1); int w=x;
    while(t[x].p>=0){
      w=realPar(x); splay(w); remVirt(x);
      addVirt(w,t[w].c[1]);
      attach(w,1,x); splay(x);
    } // 583e4a
    return w;
  } // 824a2c
  void rootify(int x){ access(x); flip(x); }
  int findRoot(int x){
    access(x);
    while(push(x),t[x].c[0]>=0)x=t[x].c[0];
    splay(x); return x;
  } // 7d2c11
  bool connected(int a,int b){return a==b||findRoot(a)==
       findRoot(b);}
  void link(int a,int b){
    rootify(a); assert(findRoot(b)!=a);
    access(b); addVirt(b,a);
  } // ef67fb
  void cut(int a,int b){
    rootify(a); access(b);
    assert(t[b].c[0]==a&&t[a].c[1]<0);
    t[b].c[0]=t[a].p=-1; pull(b);
  } // 022789
  void update(int x,T w){ access(x); t[x].w=w; pull(x); }
  T diameter(int x){ access(x); return t[x].d.d; }
}; // 0a2dc1
```

Data structures (6)

6.1 Collections

HashMap.h

Description: Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).

d77092, 7L, 2m40s

```
#include <bits/extc++.h>
// To use most bits rather than just the lowest ones:
struct chash { // large odd number for C
  const uint64_t C = ll(4e18 * acos(0)) | 71;
  ll operator()(ll x) const { return __builtin_bswap64(x*C); }
}; // cdd37e
__gnu_pbds::gp_hash_table<ll,int,chash> h({},{},{},{},{1<<16});
```

OrderedSet.h

Description: ordered/indexed set and multiset. Bad constant factor, works only in Linux. All erase/find/search functions assume the target exists.
Memory: $\mathcal{O}(N)$
Time: $\mathcal{O}(\log N)$ operations

<bits/extc++.h> //include it before any defines — b8c30a, 15L, 6m10s

```
using namespace __gnu_pbds;
template<class T, class B = null_type> using ordered_set = tree
    <T, B, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

template<class T>
struct ordered_multiset{
  ordered_set<pair<T, int>> o; int c;
  ordered_multiset():c(0){}
  unsigned order_of_key(T x){return o.order_of_key({x, -1});}
  const T* find_by_order(int p){return &(*o.find_by_order(p)).
      first;}
  void insert(T x){o.insert({x, c++});}
  void erase(T x){o.erase(o.lower_bound({x, 0}));}
  unsigned size(){return o.size();}
  const T* lower_bound(T x){return &(*o.lower_bound({x, 0})).
      first;}
  const T* upper_bound(T x){return &(*o.upper_bound({x, c})).
      first;}
}; // d4e54f
```

UnionFindRollback.h

Description: Disjoint-set data structure with undo. If undo is not needed, skip st, time() and rollback().
Usage: `int t = uf.time(); ...; uf.rollback(t);`
Time: $\mathcal{O}(\log(N))$

de4ad0, 21L, 4m50s

```
struct RollbackUF {
  vi e; vector<pii> st;
  RollbackUF(int n) : e(n, -1) {}
  int size(int x) { return -e[find(x)]; }
  int find(int x) { return e[x] < 0 ? x : find(e[x]); }
  int time() { return sz(st); }
  void rollback(int t) {
    for (int i = time(); i --> t;)
      e[st[i].first] = st[i].second;
    st.resize(t);
  } // 30bb61
  bool join(int a, int b) {
    a = find(a), b = find(b);
    if (a == b) return false;
    if (e[a] > e[b]) swap(a, b);
    st.push_back({a, e[a]});
    st.push_back({b, e[b]});
    e[a] += e[b]; e[b] = a;
    return true;
  } // 6c709f
}; // de4ad0
```

6.2 Line Queries

StaticCHT.h

Description: static CHT - add must be ordered by slope (a), queries by x.
Time: amortized $\mathcal{O}(1)$.

8331db, 33L, 7m

```
struct CHT {
  int it; vector<ll> a, b;
  CHT():it(0){}
  ll eval(int i, ll x){return a[i]*x + b[i];}
  bool useless(){
    int sz = a.size();
    int r = sz-1, m = sz-2, l = sz-3;
#warning careful with overflow!
    return (b[l] - b[r])*(a[m] - a[l]) <=
      (b[l] - b[m])*(a[r] - a[l]);
  } // 848c9b
  void add(ll A, ll B){
    if(sz(a) && A==a.back()){
      if(B<=b.back())return;
      a.pop_back(); b.pop_back();
    } // 915a14
    a.push_back(A); b.push_back(B);
    while (!a.empty()){
      if ((a.size() < 3) || !useless()) break;
      a.erase(a.end() - 2);
      b.erase(b.end() - 2);
    } // b21fc8
    it = min(it, int(a.size()) - 1);
  } // 9ef59c
  ll query(ll x){
    if (sz(a) == 0)return -oo;
    while (it+1 < a.size()){
      if (eval(it+1, x) >= eval(it, x)) it++;
      else break;
    } // 27ce3f
    return eval(it, x);
  } // adab31
}; // 8331db
```

LineContainer.h

Description: Dynamic CHT where you can add lines of the form ax+b, and query maximum values at points x. For doubles, use inf = 1/.0, floordiv(a,b) = a/b
Time: $\mathcal{O}(\log N)$

7fa0e9, 28L, 7m40s

```
ll floordiv(ll a, ll b) {
  return a / b - ((a ^ b) < 0 && a % b);
} // 7e2711
struct Line {
  mutable ll a, b, p;
  bool operator<(const Line& o) const { return a < o.a; }
  bool operator<(ll x) const { return p < x; }
}; // 469928
struct LineContainer : multiset<Line, less<>> {
  bool isect(iterator i, iterator j) {
    if (j == end()) return i->p = oo, 0;
    if (i->a == j->a) i->p = i->b > j->b ? oo : -oo;
    else i->p = floordiv(j->b - i->b, i->a - j->a);
    return i->p >= j->p;
  } // 978e9c
  void add(ll a, ll b) {
    auto k = insert({a, b, 0}), j = k++, i = j;
    while (isect(j, k)) k = erase(k);
    if (i != begin() && isect(--i, j)) isect(i, j = erase(j));
    while ((j = i) != begin() && (--i)->p >= j->p)
      isect(i, erase(j));
  } // 99e4d6
  ll query(ll x) {
    if (empty())return -oo;
    auto l = *lower_bound(x);
    return l.a * x + l.b;
  } // 22865e
}; // 35b669
```

LiChao.h

Description: Compressed Li Chao tree for max queries. Give all query x-coordinates in xs. Lines are added on [l,r].
Time: $\mathcal{O}(log^2 N)$ add, $\mathcal{O}(logN)$ query.

14b532, 44L, 11m20s

```
template<class T> struct LiChao {
  static constexpr T NINF = numeric_limits<T>::lowest();
  struct Line {
    T a=0,b=NINF;
    T operator()(T x) { return a*x+b; }
  }; // 62e8ca
  int n; vector<T> xs; vector<Line> t;
  LiChao(vector<T> _xs):xs(_xs) {
    sort(all(xs));
    xs.erase(unique(all(xs)),xs.end());
    n = sz(xs);
    t.resize(4*n);
  } // f827f2
  void add(Line v, int p, int l, int r) {
    int m=(l+r)/2;
    bool L=v(xs[l])>t[p](xs[l]), M=v(xs[m])>t[p](xs[m]);
    if (M) swap(v,t[p]);
    if (r-l==1) return;
    if (L!=M) add(v,2*p,l,m);
    else add(v,2*p+1,m,r);
```

```
  } // cc3a96
  void add(Line v, int ql, int qr, int p, int l, int r) {
    if (qr<=l or r<=ql) return;
    if (ql<=l and r<=qr) return add(v,p,l,r);
    int m=(l+r)/2;
    add(v,ql,qr,2*p,l,m); add(v,ql,qr,2*p+1,m,r);
  } // 6f709f
  void add(T a, T b, T l, T r) {
    int li=lower_bound(all(xs),l)-xs.begin();
    int ri=upper_bound(all(xs),r)-xs.begin();
    if (li<ri) add({a,b},li,ri,1,0,n);
  } // 34dfcd
  T query(T x, int i, int p, int l, int r) {
    T ans=t[p](x);
    if (r-l==1) return ans;
    int m=(l+r)/2;
    T best = i < m ? query(x,i,2*p,l,m) : query(x,i,2*p+1,m,r);
    return max(ans, best);
  } // fe58ac
  T query(T x) {
    int i=lower_bound(all(xs),x)-xs.begin();
    return query(x,i,1,0,n);
  } // 438b30
}; // 14b532
```

6.3 Range Queries

Spec.h

Description: Spec for Seg Beats. It's similar to other data structures. If id is not contexpr, declare "BeatsSpec::T BeatsSpec:: id;" globally and initialize "BeatsSpec::id" inside main
Time: varies

061722, 33L, 8m

```
struct node{int max1,max2,maxc; ll sum;};
struct BeatsSpec{ //chmin sum
  using T = node; using L = int;
  static constexpr T id = node{-oo,-oo,0,0};
  static T op(T a, T b){
    node n;
    if (a.max1 > b.max1){
      n.max1 = a.max1;
      n.max2 = max(a.max2, b.max1);
      n.maxc = a.maxc;
    } // 60044a
    else if (a.max1 == b.max1){
      n.max1 = a.max1;
      n.max2 = max(a.max2, b.max2);
      n.maxc = a.maxc+b.maxc;
    } // 89a062
    else{
      n.max1 = b.max1;
      n.max2 = max(b.max2, a.max1);
      n.maxc = b.maxc;
    } // dbf86a
    n.sum=a.sum+b.sum;
    return n;
  } // 7211b8
  static T ch(T a, L b, int l, int r){
    if (a.max1 > b)a.sum -= (ll)(a.max1-b)*a.maxc, a.max1 = b;
    return a;
  } // 54e66b
  static L cmp(L a, L b){return min(a,b);}
  static bool brk(L a, T b){return b.max1 <= a;}
  static bool tag(L a, T b){return b.max2 < a;}
  static T rev(T a){return a;} // nothing happens when the
      range is reversed
}; // 6ea242
```

6.3.1 Offline

Dist.h

Description: Disjoint Sparse Table Requires Monoid (op, id)
Memory: $\mathcal{O}(N \log N)$
Time: $\mathcal{O}(1)$

b271c7, 22L, 6m

```
#define repinv(i, a, b) for(int i = (a); i >= (b); i--)
template<class S> struct DiST{
  using T = typename S::T;
  int n, h; vector<vector<T>> t;
  int lg(unsigned x){return bit_width(x)-1;}
  DiST(vector<T> v): n(bit_ceil(v.size())),
    h(lg(n)), t(h, vector<T>(n)){
    v.resize(n, S::id);
    if (n == 1)t.pb(v);
    for(int d = 0, s = 1; d < h; d++, s *= 2)
    for(int m = s; m < n; m += 2*s){
      t[d][m] = v[m]; t[d][m-1] = v[m-1];
      rep(i, m+1, m+s)t[d][i] = S::op(t[d][i-1], v[i]);
      repinv(i, m-2, m-s)t[d][i] = S::op(v[i], t[d][i+1]);
    } // 3b44fe
  } // 2329d7
  T query(int l, int r){
    if (l==r)return t[0][l];
    int k = lg(l^r);
    return S::op(t[k][l], t[k][r]);
  } // 07c10a
}; // 8eff81
```

SparseTable.h

Description: Sparse Table Requires Idempotent Monoid S (op, id)
Memory: $\mathcal{O}(n \log n)$
Time: $\mathcal{O}(1)$ query, $\mathcal{O}(n \log n)$ build

276609, 14L, 4m

```
template<class S> struct SpTable{
  using T = typename S::T;
  int n; vector<vector<T>> tab;
  int lg(signed x){return __builtin_clz(1)-__builtin_clz(x);}
  SpTable(vector<T> v):n(sz(v)),tab(1+lg(n),vector<T>(n,S::id))
      {
    rep(i,0,n)tab[0][i] = v[i];
    rep(i,0,lg(n))rep(j,0,n-(1<<i))
      tab[i+1][j] = S::op(tab[i][j], tab[i][j+(1<<i)]);
  } // c105d7
  T query(int l, int r){
    int k = lg(++r-l);
    return S::op(tab[k][l], tab[k][r-(1<<k)]);
  } // e06689
}; // 276609
```

Mo.h

Description: Answer interval or tree path queries. Includes interval version without deletion. If values are on tree edges, change `step` to add/remove the edge (a, c) and remove the initial `add` call (but keep `in`).
Time: $\mathcal{O}\left(N\sqrt{Q}\right)$

1f1cfd, 76L, 22m30s

```
void add(int i, int end) { i+=end; }
void del(int i, int end) { i-=end; }
int calc() { return 0; }

vi mo(vector<pii> Q) {
  int L = 0, R = -1, blk = 450; // ~N/sqrt(Q)
  vi s(sz(Q)), res = s;
#define K(x) pii(x.first/blk, x.second ^ -(x.first/blk & 1))
  iota(all(s), 0);
  sort(all(s), [&](int i, int j){ return K(Q[i]) < K(Q[j]); });
  for (int qi : s) {
    auto [l, r] = Q[qi];
    while (L > l) add(--L, 0);
    while (R < r) add(++R, 1);
    while (L < l) del(L++, 0);
    while (R > r) del(R--, 1);
    res[qi] = calc();
  } // 241d86
  return res;
} // d18826

vi moTree(vector<array<int, 2>> Q, vector<vi>& ed, int root=0){
  int N = sz(ed), pos[2] = {}, blk = 350; // ~N/sqrt(Q)
  vi s(sz(Q)), res = s, I(N), L(N), R(N), in(N), par(N);
  add(root, 0), in[root] = 1;
  auto dfs = [&](int x, int p, int dep, auto& f) -> void {
    par[x] = p;
    L[x] = N;
    if (dep) I[x] = N++;
    for (int y : ed[x]) if (y != p) f(y, x, !dep, f);
    if (!dep) I[x] = N++;
    R[x] = N;
  }; // 329c88
  dfs(root, -1, 0, dfs);
#define K(x) pii(I[x[0]] / blk, I[x[1]] ^ -(I[x[0]] / blk & 1))
  iota(all(s), 0);
  sort(all(s), [&](int a, int b){ return K(Q[a]) < K(Q[b]); });
  for (int qi : s) rep(end,0,2) {
    int &a = pos[end], b = Q[qi][end], i = 0;
#define step(c) { if (in[c]) { del(a, end); in[a] = 0; } \
                  else { add(c, end); in[c] = 1; } a = c; } //
                      3839ba
    while (!(L[b] <= L[a] && R[a] <= R[b]))
      I[i++] = b, b = par[b];
    while (a != b) step(par[a]);
    while (i--) step(I[i]);
    if (end) res[qi] = calc();
  } // c880be
  return res;
} // f45ee8

vector<int> mo_no_deletion(vector<pii>& qs, int n){
  int q = sz(qs), sq = (int)sqrt(q)+1, blk = (n+sq+1)/sq;
  vector<vi> o((n+blk-1)/blk);
  rep(i,0,q)o[qs[i].first/blk].pb(i);
  for(auto& vq : o)sort(all(vq), [&](int i, int j){return qs[i
      ].second < qs[j].second;});
  vector<int> ans(q);
  rep(i,0,sz(o)){
    auto& vq = o[i];
    int l = blk*i + blk, r = l-1;
    // prepare to answer queries
    for(int qi : vq){
      auto [ql, qr] = qs[qi];
      if (qr <= l){ //if it does not extrapolate
        rep(j,ql,qr+1)add(j, 1); //solving manually
        continue;
      } // b94913
      while(r < qr)add(++r, 1);
      int ml = l; //we will move l manually
      // prepare checkpoint
      while(ml > ql)add(--ml, 0);
      ans[qi] = calc();
      // revert checkpoint: discard changes made by moving l
    } // 6569ca
  } // 09ff2d
  return ans;
} // cf095f
```

6.3.2 Online

BIT.h

Description: BIT/Fenwick Tree, requires abelian group (T, id, op, inv). For N = 10^7, allocate h(n)+1 and swap all access bit[i] for bit[h(i)]
Time: $\mathcal{O}(\log N)$.

5192eb, 20L, 5m20s

```
#define chf(a, f, b) a = f(a, b)
//int h(x){return x+(x>>10);} for N = $10^7$
template<class S> struct BIT{
  using T = typename S::T;
  int n; vector<T> bit;
  int lb(int x){return x&(-x);}
  BIT(vector<T> v):n(sz(v)),bit(1+n,S::id){
    rep(i,1,n+1){
      chf(bit[i], S::op, v[i-1]);
      if (i+lb(i)<=n)chf(bit[i+lb(i)], S::op, bit[i]);
    } // 41e113
  } // 484915
  void add(T x, int p){for(p++;p<=n;p+=lb(p))chf(bit[p],S::op,x
      );}
  T query(int l, int r){
    T lv=S::id, rv=S::id; r++;
    for(;r>=1;r-=lb(r))chf(rv,S::op,bit[r]);
    for(;l>=1;l-=lb(l))chf(lv,S::op,bit[l]);
    return S::op(rv,S::inv(lv));
  } // 990a13
}; // 4b6a8d
```

Treap.h

Description: Implicit Treap. Dynamic array with insert/erase, lazy range updates, queries and reversals. Ranges are inclusive. Operations are described at the end. S requires T,L,id,op,ch,cmp,rev (what happens to range val when reversed). ch always receives [0,len-1]. Lazy operations must commute with reversions.
Time: $\mathcal{O}(logN)$ expected per operation.

2e1a94, 84L, 21m20s

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().
    count());

template<class S> struct Treap {
  using T=typename S::T; using L=typename S::L;
  struct N {
    int l=0, r=0,c=0; unsigned y=0;
    T v=S::id, ac=S::id; L lz{};
    bool ig=1,rv=0;
    N(int C=0, T V=S::id, unsigned Y=0):c(C),y(Y),v(V),ac(V){}
  }; // a02fb4
  vector<N> t; int rt=0;
  Treap():t(1){}
  int node(T v){return t.emplace_back(1,v,rng()), sz(t)-1;}
  void apply(int p,L v){
    if(!p)return;
    t[p].v=S::ch(t[p].v,v,0,0);
    t[p].ac=S::ch(t[p].ac,v,0,t[p].c-1);
    t[p].lz=t[p].ig?v:S::cmp(t[p].lz,v); t[p].ig=0;
  } // e37416
  void flip(int p){
    if(!p)return;
    swap(t[p].l,t[p].r);
    t[p].ac=S::rev(t[p].ac);
    t[p].rv^=1;
  } // b92f4f
  void push(int p){
    if(t[p].rv) flip(t[p].l),flip(t[p].r),t[p].rv=0;
    if(!t[p].ig)apply(t[p].l,t[p].lz),apply(t[p].r,t[p].lz),t[p
        ].ig=1;
  } // 42a806
  void pull(int p){
    t[p].c=t[t[p].l].c+t[t[p].r].c+1;
    t[p].ac=S::op(S::op(t[t[p].l].ac,t[p].v),t[t[p].r].ac);
  } // 1bc654
  pair<int,int> split(int p,int k){ // first k go left
    if(!p)return {};
    push(p);
    if(t[t[p].l].c>=k){
      auto [a,b]=split(t[p].l,k);
      t[p].l=b; pull(p); return {a,p};
    } // f522e3
    auto [a,b]=split(t[p].r,k-t[t[p].l].c-1);
    t[p].r=a; pull(p); return {p,b};
  } // a5d263
  int merge(int a,int b){
    if(!a||!b)return a?a:b;
    if(t[a].y>t[b].y){
      push(a); t[a].r=merge(t[a].r,b); pull(a); return a;
    } // 77ec5b
    push(b); t[b].l=merge(a,t[b].l); pull(b); return b;
  } // c5d1b2
}; // 6195b6

/* Additional optional operations:
  Treap(vector<T> a={}):t(1){
    t.reserve(sz(a)+1);
    for(T x:a)rt=merge(rt,node(x));
  } // 6d6612
  array<int,3> cut(int l,int r){
    auto [a,c]=split(rt,r+1);
    auto [x,b]=split(a,l);
    return {x,b,c};
  } // dbc1d0
  void join(array<int,3> a){rt=merge(merge(a[0],a[1]),a[2]);}
  void insert(int p,T v){
    auto [a,b]=split(rt,p);
    rt=merge(merge(a,node(v)),b);
  } // 46fa14
  void erase(int l,int r){
    auto a=cut(l,r);
    rt=merge(a[0],a[2]);
  } // bde6f5
  void update(L v,int l,int r){
    auto a=cut(l,r); apply(a[1],v); join(a);
  } // 0f7e99
  void reverse(int l,int r){
    auto a=cut(l,r); flip(a[1]); join(a);
  } // 2425ca
  T query(int l,int r){
    auto a=cut(l,r);
    T ans=t[a[1]].ac; join(a); return ans;
  } // a1eeba
  T get(int p){return query(p,p);}
  int size(){return t[rt].c;}
*/
```

SQRT Decomposition

SqrtFastUpdate.h

Description: Sqrt decomposition for many point updates and few queries. update(i,x) applies a[i] = op(a[i],x). S must be a commutative monoid.
Memory: $\mathcal{O}(N)$.
Time: $\mathcal{O}(1)$ update, $\mathcal{O}\left(\sqrt{N}\right)$ query.

5e07c4, 19L, 4m20s

```
template<class S> struct SqrtFastUpdate {
  using T=typename S::T;
  int n, B; vector<T> a, b;
  SqrtFastUpdate(vector<T> A):n(sz(A)),B(max(1,(int)sqrt(n))),
    a(move(A)),b((n+B-1)/B,S::id) {
    rep(i,0,n)b[i/B]=S::op(b[i/B],a[i]);
  } // 9dce63
  void update(T x, int i){
    a[i]=S::op(a[i],x);
    b[i/B]=S::op(b[i/B],x);
  } // 60cddf
  T query(int l,int r){
    T x=S::id;
    while(l<=r && l%B)x=S::op(x,a[l++]);
    while(l+B-1<=r)x=S::op(x,b[l/B]),l+=B;
    while(l<=r)x=S::op(x,a[l++]);
    return x;
  } // 1b9392
}; // 5e07c4
```

SqrtFastQuery.h

Description: Sqrt decomposition for many queries and few point updates. S must be a group.
Memory: $\mathcal{O}(N)$.
Time: $\mathcal{O}\left(\sqrt{N}\right)$ set, $\mathcal{O}(1)$ query.

e0a07a, 27L, 6m40s

```
template<class S> struct SqrtFastQuery {
  using T=typename S::T;
  int n,B,nb; vector<T> a,pre,bpre;
  SqrtFastQuery(vector<T> A):n(sz(A)),B(max(1,(int)sqrt(n))),
    nb((n+B-1)/B),a(move(A)),pre(n),bpre(nb) {
    rep(k,0,nb){
      T x=S::id;
      rep(i,k*B,min(n,(k+1)*B))pre[i]=x=S::op(x,a[i]);
      bpre[k]=S::op(k?bpre[k-1]:S::id,x);
    } // 4e0a4a
  } // a35fa4
  void set(T x, int i){
    a[i]=x;
    int k=i/B, e=min(n,(k+1)*B);
    T y=i%B ? pre[i-1] : S::id;
    rep(j,i,e)pre[j]=y=S::op(y,a[j]);
    rep(j,k,nb)bpre[j]=S::op(j?bpre[j-1]:S::id,pre[min(n,(j+1)*
        B)-1]);
  } // 4a9250
  T query(int l,int r){
    auto pref=[&](int i){
      if(i<0)return S::id;
      int k=i/B;
      return S::op(k?bpre[k-1]:S::id,pre[i]);
    }; // ae85c1
    return S::op(S::inv(pref(l-1)),pref(r));
  } // dfdedb
}; // e0a07a
```

Segment Tree

SegmentTree.h

Description: Iterative SegTree
Time: $\mathcal{O}(\log N)$

6263bc, 17L, 3m40s

```
template<class S> struct SegTree{
  using T = typename S::T;
  int n; vector<T> seg;
  SegTree(int s):n(s),seg(2*n,S::id){}
  void update(T x, int i){
    for(seg[i+=n]=x;i/=2;)
      seg[i] = S::op(seg[2*i], seg[2*i+1]);
  } // 781838
  T query(int l, int r){
    T vl=S::id, vr=S::id;
    for(l+=n,r+=n+1;l<r;l/=2,r/=2){
      if (l&1)vl = S::op(vl, seg[l++]);
      if (r&1)vr = S::op(seg[--r], vr);
    } // ea3c70
    return S::op(vl, vr);
```

```
  } // 524637
}; // 6263bc
```

LazySegmentTree.h

Description: Lazy Seg. Can be transformed into Seg Beats by uncommenting conditions.
Time: $\mathcal{O}(\log N * (ch + cmp))$.

1ad942, 30L, 11m20s

```
template<class S> struct SegBeats{
  using T = typename S::T; using L = typename S::L;
  int n; vector<T> seg; vector<L> lz; vector<bool> ig;
  SegBeats(unsigned s):n(bit_ceil(s)),seg(2*n,S::id),lz(2*n),ig
      (2*n,1){}
  void apply(int p, L v, int l, int r){
    seg[p] = S::ch(seg[p],v,l,r-1);
    if (r-l>1)lz[p] = ig[p] ? v : S::cmp(lz[p], v), ig[p] = 0;
  } // ada588
  int prop(int p, int l, int r){
    int m = (l+r)/2;
    if (not ig[p])ig[p] = 1,
      apply(2*p, lz[p], l, m), apply(2*p+1, lz[p], m, r);
    return m;
  } // 3cadda
  void update(L v, int l, int r){return update(v,l,r+1,1,0,n);}
  void update(L v, int lq, int rq, int no, int lx, int rx){
    if (rq <= lx or rx <= lq /*or S::brk(v,seg[no])*/)return;
    if (lq <= lx and rx <= rq /*and S::tag(v,seg[no])*/)return
        apply(no, v, lx, rx);
    int mx = prop(no,lx,rx);
    update(v,lq,rq,2*no,lx,mx); update(v,lq,rq,2*no+1,mx,rx);
    seg[no] = S::op(seg[2*no],seg[2*no+1]);
  } // f11ac8
  T query(int l, int r){return query(l,r+1,1,0,n);}
  T query(int lq, int rq, int no, int lx, int rx){
    if (rq <= lx or rx <= lq)return S::id;
    if (lq <= lx and rx <= rq)return seg[no];
    int mx = prop(no,lx,rx);
    return S::op(query(lq,rq,2*no,lx,mx),query(lq,rq,2*no+1,mx,
        rx));
  } // 0ed921
}; // 1ad942
```

SparseSeg.h

Description: Sparse Seg (I is index type)
Time: $\mathcal{O}(\log N)$

31cd4a, 32L, 8m40s

```
template<class I, class S> struct SparseSeg{
  using T = typename S::T;
  I n; vector<int> lc, rc; vector<T> val;
  int new_node(){
    int res = sz(lc);
    lc.pb(0); rc.pb(0); val.pb(S::id);
    return res;
  } // b4e987
  SparseSeg(I s): n(s){ new_node(); new_node(); }
  void update(T x, I p){return update(x, p, 1, 0, n);}
  void update(T x, I p, int i, I lx, I rx){
    if (rx-lx==1){val[i] = x; return;}
    I mx = (lx+(rx-lx)/2);
    int li = lc[i], ri = rc[i];
    if (p < mx){
      if (not li)li = lc[i] = new_node();
      update(x, p, li, lx, mx);
    } // 452d3b
    else{
      if (not ri)ri = rc[i] = new_node();
      update(x, p, ri, mx, rx);
    } // 9a7652
    val[i] = S::op(val[li], val[ri]);
  } // 1ed0c3
  T query(I l, I r){return query(l, r+1, 1, 0, n);}
  T query(I l, I r, int i, I lx, I rx){
    if (r <= lx or rx <= l)return S::id;
    if (l <= lx and rx <= r)return val[i];
    I mx = (lx+(rx-lx)/2);
    return S::op(query(l, r, lc[i], lx, mx), query(l, r, rc[i],
        mx, rx));
  } // 61def7
}; // 31cd4a
```

LazySparseSeg.h

Description: Lazy Sparse Seg, similar to Lazy Seg (I is index type). Queries also create nodes
Time: $\mathcal{O}(\log N * (ch + cmp))$.

a37732, 39L, 12m50s

```
template<class I, class S> struct LazySparseSeg{
  using T = typename S::T; using L = typename S::L;
  I n; vector<int> lc, rc; vector<bool> ig;
  vector<T> val; vector<L> lz;
  int new_node(){
    int res = sz(lc);
    lc.pb(0); rc.pb(0); ig.pb(1);
    val.pb(S::id); lz.pb(L());
    return res;
  } // 81c523
  LazySparseSeg(I s): n(s){ new_node(); new_node(); }
  void apply(int i, L v, I lx, I rx){
    val[i] = S::ch(val[i], v, lx, rx-1);
    if (rx-lx>1)lz[i] = ig[i] ? v : S::cmp(lz[i], v), ig[i] =
        0;
  } // 0e5514
  I prop(int i, I lx, I rx){
    I mx = (lx+(rx-lx)/2);
    if (!lc[i])lc[i] = new_node(), rc[i] = new_node();
    if (not ig[i])ig[i] = 1,
      apply(lc[i], lz[i], lx, mx), apply(rc[i], lz[i], mx, rx);
    return mx;
  } // b3b231
  void update(L v, I l, I r){return update(v, l, r+1, 1, 0, n);
      }
  void update(L v, I l, I r, int i, I lx, I rx){
    if (r <= lx or rx <= l)return;
    if (l <= lx and rx <= r)return apply(i, v, lx, rx);
    I mx = prop(i, lx, rx);
    int li = lc[i], ri = rc[i];
    update(v, l, r, li, lx, mx); update(v, l, r, ri, mx, rx);
    val[i] = S::op(val[li], val[ri]);
  } // bf0ca8
  T query(I l, I r){return query(l, r+1, 1, 0, n);}
  T query(I l, I r, int i, I lx, I rx){
    if (r <= lx or rx <= l)return S::id;
    if (l <= lx and rx <= r)return val[i];
    I mx = prop(i, lx, rx);
    return S::op(query(l, r, lc[i], lx, mx), query(l, r, rc[i],
        mx, rx));
  } // 1ef417
}; // a37732
```

PersistentSeg.h

Description: Persistent Segment Tree. Call "init" to get the root!
Time: $\mathcal{O}(\log N)$

807b80, 26L, 9m20s

```
template<class I, class S> struct PersistentSeg{
  using T = typename S::T;
  I n; vector<int> lc, rc; vector<T> val;
  int new_node(int l, int r, T x){
    int res = sz(lc);
    lc.pb(l); rc.pb(r); val.pb(x);
    return res;
  } // eb1f5d
  int new_node(int l, int r){return new_node(l, r, S::op(val[l
      ], val[r]));}
  PersistentSeg(){ new_node(0,0,S::id); }
  int init(I s){return n = s, new_node(0,0,S::id);}
  int update(T x, I p, int root){return update(x, p, root, 0, n
      );}
  int update(T x, I p, int i, I lx, I rx){
    if (rx-lx==1)return new_node(0,0,x);
    I mx = lx + (rx - lx) / 2;
    if (p < mx)return new_node(update(x, p, lc[i], lx, mx), rc[
        i]);
    else return new_node(lc[i], update(x, p, rc[i], mx, rx));
  } // 7644dd
  T query(I l, I r, int root){return query(l, r+1, root, 0, n);
      }
  T query(I l, I r, int i, I lx, I rx){
    if (r <= lx or rx <= l)return S::id;
    if (l <= lx and rx <= r)return val[i];
    I mx = (lx+(rx-lx)/2);
    return S::op(query(l, r, lc[i], lx, mx), query(l, r, rc[i],
        mx, rx));
  } // 61def7
}; // 807b80
```

LazyPersistentSeg.h

Description: Persistent Lazy Sparse Segment Tree. Call "init" to get the root! Queries also create nodes.
Time: $\mathcal{O}(\log N * (ch + cmp))$

47805d, 46L, 14m20s

```
template<class I, class S> struct LazyPersistentSeg{
  using T = typename S::T;
  using L = typename S::L;
  I n; vector<int> lc, rc; vector<bool> ig;
  vector<T> val; vector<L> lz;
  int new_node(int l, int r){
    int res = sz(lc);
    lc.pb(l); rc.pb(r); ig.pb(1);
    val.pb(S::op(val[l], val[r])); lz.pb(L());
    return res;
  } // e66e23
  LazyPersistentSeg(){
    lc.pb(0); rc.pb(0); ig.pb(1);
    val.pb(S::id); lz.pb(L());
  } // ccb55c
  int init(I s){return n = s, new_node(0, 0);}
  int lazy_clone(int i, L v, I lx, I rx){
    int ni = new_node(lc[i], rc[i]);
    lz[ni] = ig[i] ? v : S::cmp(lz[i], v);
    ig[ni] = 0; val[ni] = S::ch(val[i], v, lx, rx-1);
    return ni;
  } // 56e9d8
  I prop(int i, I lx, I rx){
    I mx = lx + (rx - lx) / 2;
    if (not ig[i]){
      ig[i] = 1; if (rx - lx > 1)
      lc[i] = lazy_clone(lc[i], lz[i], lx, mx),
      rc[i] = lazy_clone(rc[i], lz[i], mx, rx);
    } // a69d31
    return mx;
  } // 827037
  int update(L v, I l, I r, int root){return update(v, l, r+1,
      root, 0, n);}
  int update(L v, I l, I r, int i, I lx, I rx){
    if (r <= lx or rx <= l)return i;
    if (l <= lx and rx <= r)return lazy_clone(i, v, lx, rx);
```

```
    I mx = prop(i, lx, rx);
    return new_node(update(v, l, r, lc[i], lx, mx), update(v, l
        , r, rc[i], mx, rx));
  } // 49ce75
  T query(I l, I r, int root){return query(l, r+1, root, 0, n);
      }
  T query(I l, I r, int i, I lx, I rx){
    if (r <= lx or rx <= l)return S::id;
    if (l <= lx and rx <= r)return val[i];
    I mx = prop(i, lx, rx);
    return S::op(query(l, r, lc[i], lx, mx), query(l, r, rc[i],
         mx, rx));
  } // 1ef417
}; // 47805d
```

6.3.3 MDRQDS

MPsum.h

Description: Multidimensional Psum Requires Abelian Group (op, inv, id)

Memory: $\mathcal{O}\left(N^D\right)$

Time: $\mathcal{O}\left(2^D\right)$

ccbcbd, 20L, 6m10s

```
#define MAs template<class... As>
template<int D, class S> struct MPsum{
  using T = typename S::T;
  int n; vector<MPsum<D-1, S>> v;
  MAs MPsum(int s, As... ds):n(s+1),v(n,MPsum<D-1, S>(ds...)){}
  MAs void set(T x, int p, As... ps){v[p+1].set(x, ps...);}
  void push(MPsum& p){rep(i, 1, n)v[i].push(p.v[i]);}
  void init(){rep(i, 1, n)v[i].init(),v[i].push(v[i-1]);}
  MAs T query(int l, int r, As... ps){
    return S::op(v[r+1].query(ps...),S::inv(v[l].query(ps...)))
        ;
  } // eac6a8
}; // f3cfdf
template<class S> struct MPsum<0, S>{
  using T = typename S::T;
  T val=S::id;
  void set(T x){val=x;}
  void push(MPsum& a){val=S::op(a.val,val);}
  void init(){}
  T query(){return val;}
}; // ee6e4e
```

MSpTable.h

Description: Multidimensional Sparse Table Requires commutative idempotent monoid (T, id, op)

Memory: $\mathcal{O}\left((N \log N)^D\right)$

Time: $\mathcal{O}\left(2^D\right)$

d3247e, 32L, 9m20s

```
#define MAs template<class...As>
template<int D, class S> struct MSpTable{
  using T = typename S::T;
  using isp = MSpTable<D-1, S>;
  int lg(signed x){return __builtin_clz(1)-__builtin_clz(x);}
  int n; vector<vector<isp>> tab;
  MAs MSpTable(int s, As... ds):n(s),
  tab(1+lg(n),vector<isp>(n,isp(ds...))){}
  MAs void set(T x, int p, As... ps){tab[0][p].set(x, ps...);}
  void join(MSpTable& a, MSpTable& b){
    rep(i, 0, 1+lg(n))rep(j, 0, n)
      tab[i][j].join(a.tab[i][j], b.tab[i][j]);
  } // 96d587
  void init(){
    rep(i, 0, n)tab[0][i].init();
    rep(i, 0, lg(n))rep(j, 0, n-(1<<i))
      tab[i+1][j].join(tab[i][j], tab[i][j+(1<<i)]);
  } // dc476b
  MAs T query(int l, int r, As... ps){
    int k = lg(r-l+1); r+=1-(1<<k);
    return S::op(tab[k][l].query(ps...),tab[k][r].query(ps...))
        ;
  } // b8ce4c
}; // 0764a3

template<class S> struct MSpTable<0, S>{
  using T = typename S::T;
  T val=S::id;
  void set(T x){val=x;}
  void join(MSpTable& a, MSpTable& b){val=S::op(a.val,b.val);}
  void init(){}
  T query(){return val;}
}; // e7585b
```

MSegTree.h

Description: Multidimensional SegTree Requires commutative Monoid (op, id)

Memory: $\mathcal{O}\left(N^D\right)$

Time: $\mathcal{O}\left((\log N)^D\right)$

e335e5, 26L, 7m40s

```
#define MAs template<class... As>
template<int D, class S> struct MSegTree{
  using T = typename S::T;
  int n; vector<MSegTree<D-1, S>> seg;
  MAs MSegTree(int s, As... ds):n(s),seg(2*n, MSegTree<D-1, S>(
      ds...)){}
  MAs T get(int p, As... ps){return seg[p+n].get(ps...);}
  MAs void update(T x, int p, As... ps){
    seg[p+=n].update(x, ps...);
    while(p/=2)seg[p].update(S::op(seg[2*p].get(ps...),seg[2*p
        +1].get(ps...)), ps...);
  } // 2c8b52
  MAs T query(int l, int r, As... ps){
    T lv=S::id,rv=S::id;
    for(l+=n,r+=n+1;l<r;l/=2,r/=2){
      if (l&1)lv = S::op(lv,seg[l++].query(ps...));
      if (r&1)rv = S::op(seg[--r].query(ps...),rv);
    } // 1569b6
    return S::op(lv,rv);
  } // bc7474
}; // da06ff
template<class S> struct MSegTree<0, S>{
  using T = typename S::T;
  T val=S::id;
  T get(){return val;}
  void update(T x){val=x;}
  T query(){return val;}
}; // 50402e
```

CMBIT.h

Description: Compressed multidimensional BIT. All update points must be given beforehand. Queries are inclusive. K here is the number of update points.

Memory: $\mathcal{O}\left(Klog^(D-1)K\right)$.

Time: $\mathcal{O}\left(log^D K\right)$ update, $\mathcal{O}\left((2logK)^D\right)$ query.

a0f382, 35L, 11m

```
#define MAs template<class... A>
template<class I, int D, class S> struct CMBIT {
  using T=typename S::T; using P=array<I,D-1>;
  int n; vector<I> x; vector<CMBIT<I,D-1,S>> ft;
  int ilb(I i){return (int)(lower_bound(all(x),i)-x.begin());}
  int irb(I i){return (int)(upper_bound(all(x),i)-x.begin());}
  CMBIT(vector<array<I, D>> p){
    sort(all(p)); p.erase(unique(all(p)),p.end());
    x.reserve(sz(p));
    for(auto &a:p) if(x.empty() || x.back()!=a[0])x.pb(a[0]);
    vector<vector<P>> ips(n = sz(x) + 1);
    for(auto &a:p){
      P b; rep(j,1,D) b[j-1]=a[j];
      for(int i=ilb(a[0])+1;i<n;i+=i&-i)ips[i].pb(b);
    } // 5e2514
    ft.reserve(n); for(auto &y:ips) ft.eb(move(y));
  } // 43e0d7
  MAs void update(T v, I p, A... ps){
    for(int i=ilb(p)+1;i<n;i+=i&-i)ft[i].update(v,ps...);
  } // b9d0b2
  MAs T query(I l, I r, A... ps){
    T a=S::id,b=S::id;
    for(int R = irb(r);R;R-=R&-R)b=S::op(b,ft[R].query(ps...));
    for(int L = ilb(l);L;L-=L&-L)a=S::op(a,ft[L].query(ps...));
    return S::op(b,S::inv(a));
  } // 963674
}; // ac3671

template<class I, class S> struct CMBIT<I,0,S> {
  using T=typename S::T;
  T v=S::id;
  CMBIT(vector<array<I,0>> ig) {}
  void update(T x){v=S::op(v,x);}
  T query(){return v;}
}; // ac559d
```

CMSEG.h

Description: Compressed multidimensional SegTree. All update points must be given beforehand. Set updates and inclusive queries.

Memory: $\mathcal{O}\left(Klog^(D-1)K\right)$.

Time: $\mathcal{O}\left(log^D K\right)$ update/query.

7de0c3, 43L, 12m50s

```
#define MAs template<class... A>
template<class I, int D, class S> struct CMSEG {
  using T=typename S::T; using P=array<I,D-1>;
  int n; vector<I> x; vector<CMSEG<I,D-1,S>> seg;
  int ilb(I i){return lower_bound(all(x),i)-x.begin();}
  int irb(I i){return upper_bound(all(x),i)-x.begin();}
  CMSEG(vector<array<I,D>> p){
    sort(all(p)); p.erase(unique(all(p)),p.end());
    x.reserve(sz(p));
    for(auto &a:p) if(x.empty() || x.back()!=a[0])x.pb(a[0]);
    vector<vector<P>> ips(2*(n=sz(x)));
    for(auto &a:p){
      P b; rep(j,1,D)b[j-1]=a[j];
      for(int i=ilb(a[0])+n;i;i/=2)ips[i].pb(b);
    } // b4a19f
    seg.reserve(2*n); for(auto &y:ips)seg.eb(move(y));
  } // daffca
  MAs T get(I p,A... ps){
    int i=ilb(p);
    return i<n && x[i]==p ? seg[i+n].get(ps...) : S::id;
  } // 6aa8eb
  MAs void update(T v,I p,A... ps){
    int i=ilb(p)+n; seg[i].update(v,ps...);
    while(i/=2)seg[i].update(S::op(seg[2*i].get(ps...), seg[2*i
        +1].get(ps...)),ps...);
  } // d8166c
  MAs T query(I l,I r,A... ps){
    T lv=S::id,rv=S::id;
    for(int L=ilb(l)+n,R=irb(r)+n;L<R;L/=2,R/=2){
      if(L&1)lv=S::op(lv,seg[L++].query(ps...));
      if(R&1)rv=S::op(seg[--R].query(ps...),rv);
    } // baadc3
    return S::op(lv,rv);
```

```
  } // bfe628
}; // 27a5de

template<class I, class S> struct CMSEG<I,0,S> {
  using T=typename S::T;
  T v=S::id;
  CMSEG(vector<array<I,0>> ig){}
  T get(){return v;}
  void update(T x){v=x;}
  T query(){return v;}
}; // eea5b0
```

Strings (7)

Hashing.h

Description: String hashing (multiple mods and 2^{32})
Memory: $\mathcal{O}(n)$
Time: $\mathcal{O}(1)$ query, $\mathcal{O}(n)$ build

fa5518, 23L, 8m10s

```
typedef uint64_t ull;
template<int M, class B> struct A {
  int x; B b; A(int x=0) : x(x), b(x) {}
  A(int x, B b) : x(x), b(b) {}
  A operator+(A o) const {int y = x+o.x; return{y - (y>=M)*M, b
      +o.b};}
  A operator-(A o) const {int y = x-o.x; return{y + (y< 0)*M, b
      -o.b};}
  A operator*(A o) const { return {(int)((ll)x*o.x % M), b*o.b}
      ; }
  explicit operator ull() const { return x ^ (ull) b << 21; }
}; // e03276
typedef A<1000000007, A<1000000009, unsigned>> H;
static int C = 9548; // initialize to a number less than MOD or
     random
struct HashInterval {
  int n; vector<H> ha, pw;
  template<typename S>
  HashInterval(const S & str) : n(sz(str)), ha(n+1), pw(n+1) {
    pw[0] = 1;
    rep(i,0,n)
      ha[i+1] = ha[i] * C + str[i],
      pw[i+1] = pw[i] * C;
  } // 185a86
  H query(int a, int b) { return ha[b+1] - ha[a] * pw[b-a+1]; }
  H queryI(int a, int b) { return query(n - b - 1, n - a - 1);
      }
}; // 1a81bd
```

KMP.h

Description: KMP automaton
Memory: $\mathcal{O}(N)$
Time: $\mathcal{O}(N)$ build, $\mathcal{O}(N+M)$ sequential querying

8f8450, 19L, 5m20s

```
struct KMP {
  string P; int n; vector<int> nb;
  KMP(string& p) : P(p), n((int)P.size()), nb(n+1) {
    for(int k = 1; k < n; k++) nb[k+1] = nxt(nb[k], P[k]);
  } // ca6dc8
  int nxt(int i, char c){
    for(; i; i = nb[i])if (i < n and P[i]==c)return i+1;
    return P[0]==c;
  } // 2b99e2
  vector<vector<int>> dfa;
    void build_dfa(){ // uses a-z
    dfa.assign(n+1, vector<int>(26));
        dfa[0][P[0]-'a'] = 1;
        for(int k = 1; k <= n; k++)
            for(int c = 0; c < 26; c++)
                if (k < n and P[k] == 'a'+c) dfa[k][c] = k+1;
                else dfa[k][c] = dfa[nb[k]][c];
    } // f47d83
}; // 8f8450
```

Zfunc.h

Description: z[i] computes the length of the longest common prefix of s[i:] and s, except z[0] = 0. (abacaba -> 0010301)
Time: $\mathcal{O}(n)$

ee09e2, 9L, 2m

```
vi Z(const string& S) {
  vi z(sz(S)); int l = -1, r = -1;
  rep(i,1,sz(S)) {
    z[i] = i >= r ? 0 : min(r - i, z[i - l]);
    while (i + z[i] < sz(S) && S[i + z[i]] == S[z[i]]) z[i]++;
    if (i + z[i] > r) l = i, r = i + z[i];
  } // 44be47
  return z;
} // ee09e2
```

Manacher.h

Description: For each position in a string, computes p[0][i] = half length of longest even palindrome around pos i, p[1][i] = longest odd (half rounded down).
Time: $\mathcal{O}(N)$

e7ad79, 11L, 3m

```
array<vi, 2> manacher(const string& s) {
  int n = sz(s); array<vi,2> p = {vi(n+1), vi(n)};
  rep(z,0,2) for (int i=0,l=0,r=0; i < n; i++) {
    int t = r-i+!z;
    if (i<r) p[z][i] = min(t, p[z][l+t]);
    int L = i-p[z][i], R = i+p[z][i]-!z;
    while (L>=1 && R+1<n && s[L-1] == s[R+1]) p[z][i]++, L--, R
        ++;
    if (R>r) l=L, r=R;
  } // a843d3
  return p;
} // e7ad79
```

MinRotation.h

Description: Finds the lexicographically smallest rotation of a string.
Usage: `rotate(v.begin(), v.begin()+minRotation(v), v.end());`
Time: $\mathcal{O}(N)$

d07a42, 8L, 1m40s

```
int minRotation(string s) {
  int a=0, N=sz(s); s += s;
  rep(b,0,N) rep(k,0,N) {
    if (a+k == b || s[a+k] < s[b+k]) {b += max(0, k-1); break;}
    if (s[a+k] > s[b+k]) { a = b; break; }
  } // 9374b1
  return a;
} // d07a42
```

Automaton.h

Description: Suffix automaton with parameterized transition table (can be array or map)
Memory: $\mathcal{O}(n * |G|)$
Time: $\mathcal{O}(n * accesstoG)$ build

70eb49, 30L, 7m20s

```
template<class C, class G> struct Automaton{
  vector<G> dag;
  vector<int> lnk, len, cnt;
  int last;
  template<class S> Automaton(S& str):
    dag(2),lnk(2,0),len(2,0),cnt(2,0),last(1){
    for(C& c : str)add(c); //can do .reserve(2*n+2)
  } // cd5999
  int new_node(int l, int is_pref){
    dag.pb(G()); lnk.pb(0); len.pb(l); cnt.pb(is_pref);
    return sz(dag)-1;
  } // 0814f6
  void add(C& c){
    int cur = new_node(len[last]+1, 1), p = last;
    while(p and not dag[p][c])dag[p][c] = cur, p = lnk[p];
    if (not p)lnk[cur] = 1;
    else {
      int q = dag[p][c];
      if (len[p]+1 == len[q])lnk[cur] = q;
      else{
        int clone = new_node(len[p]+1, 0);
        dag[clone] = dag[q];
        lnk[clone] = lnk[q];
        lnk[q] = lnk[cur] = clone;
        while(p and dag[p][c] == q)dag[p][c]=clone, p=lnk[p];
      } // 270a8a
    } // 9e6a19
    last = cur;
  } // b1dc88
}; // 70eb49
```

Aho.h

Description: Aho-Corasick
Memory: $\mathcal{O}(n * 26)$
Time: $\mathcal{O}(n)$ build

ccf3a0, 47L, 9m40s

```
const int K = 26;
struct Vertex {
    int next[K];
    bool output = false;
    int p = -1;
    char pch;
    int link = -1;
    int go[K];

    Vertex(int _p=-1, char ch='$') : p(_p), pch(ch) {
        fill(begin(next), end(next), -1);
        fill(begin(go), end(go), -1);
    } // 2636d9
}; // 0c589a

vector<Vertex> t(1);

void add_string(string const& s) {
    int v = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (t[v].next[c] == -1) {
            t[v].next[c] = sz(t);
            t.eb(v, ch);
        } // 2abd5c
        v = t[v].next[c];
    } // fc1e0d
    t[v].output = true;
} // 1bb4a6

int go(int v, char ch);
int get_link(int v) {
    if (t[v].link == -1) {
        if (v == 0 || t[v].p == 0) t[v].link = 0;
        else t[v].link = go(get_link(t[v].p), t[v].pch);
    } // e9856f
    return t[v].link;
} // 1e38ab

int go(int v, char ch) {
    int c = ch - 'a';
```

```
    if (t[v].go[c] == -1) {
        if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
        else t[v].go[c] = v == 0 ? 0 : go(get_link(v), ch);
    } // 202af4
    return t[v].go[c];
} // e442d8
```

DP (8)

SOS.h

Description: SOS DP. Calculates sum over subsets and supersets
Memory: $\mathcal{O}(N)$.
Time: $\mathcal{O}(N \log N)$.

e3a406, 8L, 1m50s

```
pair<vi, vi> sos(int l, int n, vi& a){ // n = 1 << l
  auto sub = a;
  auto sup = a;
  rep(b,0,l)rep(i,0,n)
    if (i & (1 << b))sub[i] += sub[i^(1<<b)];
    else sup[i] += sup[i|(1<<b)];
  return {sub, sup};
} // e3a406
```

We don't know DP optimizations yet :/

Various (9)

9.1 Random

RNG.h

Description: RNGs
Time: $\mathcal{O}(1)$

940745, 13L, 4m20s

```
mt19937 rng(chrono::steady_clock::now().time_since_epoch().
    count()); // mt19937_64
// uniform_int_distribution<int> distribution(1,n);
// int num = distribution(rng); // num no range [1, n]
// shuffle(vec.begin(), vec.end(), rng); // shuffle

using ull = unsigned long long;
ull mix(ull o){
    o+=0x9e3779b97f4a7c15;
    o=(o^(o>>30))*0xbf58476d1ce4e5b9;
    o=(o^(o>>27))*0x94d049bb133111eb;
    return o^(o>>31);
} // bc6211
ull hash(pii a) {return mix(a.first ^ mix(a.second));}
```

9.2 Ternary Search

GoldenSectionSearch.h

Description: Finds the argument minimizing the function f in the interval $[a, b]$ assuming f is unimodal on the interval, i.e. has only one local minimum and no local maximum. The maximum error in the result is *eps*. Works equally well for maximization with a small change in the code. See TernarySearch.h in for a discrete version.
Usage: `double func(double x) { return 4+x+.3*x*x; }`
`double xmin = gss(-1000,1000,func);`
Time: $\mathcal{O}(\log((b-a)/\epsilon))$

d7b114, 17L, 4m40s

```
// It is important for r to be precise,
// otherwise we don't necessarily maintain the inequality a <
    x1 < x2 < b.
template<class F>
double gss(double a, double b, F f) {
  double r = (sqrt(5)-1)/2, eps = 1e-7;
  double x1 = b - r*(b-a), x2 = a + r*(b-a);
  double f1 = f(x1), f2 = f(x2);
  while (b-a > eps)
    if (f1 < f2) { //change to > to find maximum
      b = x2; x2 = x1; f2 = f1;
      x1 = b - r*(b-a); f1 = f(x1);
    } else { // 4513d0
      a = x1; x1 = x2; f1 = f2;
      x2 = a + r*(b-a); f2 = f(x2);
    } // 2fe74a
  return a;
} // ed0140
```

TernarySearch.h

Description: Find the smallest i in $[a, b]$ that maximizes $f(i)$, assuming that $f(a) < \ldots < f(i) \geq \ldots \geq f(b)$. To reverse which of the sides allows non-strict inequalities, change the $<$ marked with (A) to $<=$, and reverse the loop at (B). To minimize f, change it to $>$, also at (B).
Usage: `int ind = ternSearch(0,n-1,[&](int i){return a[i];});`
Time: $\mathcal{O}(\log(b-a))$

afbac9, 9L, 2m

```
template<class F> int ternSearch(int a, int b, F f) {
  while (b - a >= 5) {
    int mid = (a + b) / 2;
    if (f(mid) < f(mid+1)) a = mid; // (A)
    else b = mid+1;
  } // ce7859
  rep(i,a+1,b+1) if (f(a) < f(i)) a = i; // (B)
  return a;
} // afbac9
```

9.3 Optimization tricks

9.3.1 Bit hacks

- `x & -x` is the least bit in `x`.
- `for (int x = m; x; ) { --x &= m; ... }` loops over all subset masks of `m` (except `m` itself).
- `c = x&-x, r = x+c; (((r^x) >> 2)/c) | r` is the next number after `x` with the same number of bits set.

9.3.2 Pragmas

- `#pragma GCC optimize("Ofast")` will make GCC auto-vectorize loops and aggressively optimize floating points.
- `#pragma GCC target("avx2")` can improve performance of vectorized code, but causes crashes on old machines.
- `#pragma GCC target("bmi,bmi2,lzcnt,popcnt")` are good for bitsets.